



NATIONAL UNIVERSITY OF SINGAPORE

EE4415: Integrated Digital Design

Lab 1 Part 2

Large Language Model and Low Precision ALU

A report submitted to

Professor Dr. Alioto Massimo

College of Design and Engineering

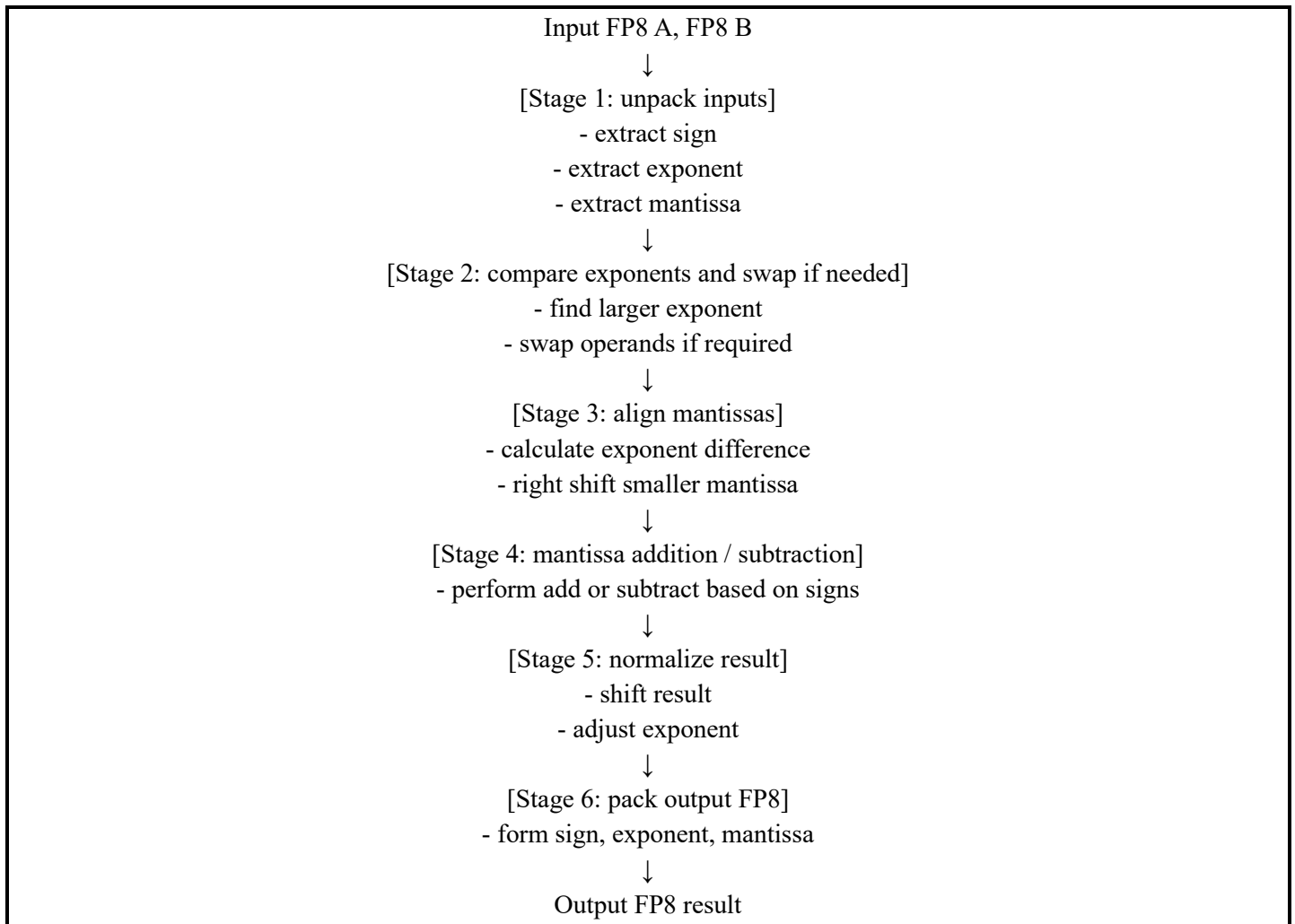
Table of Contents

EE4415 Lab 2 – Large Language Model Report	3
1. FP8 Adder Part 1:	3
2. FP8 Adder Part2:	10
3. FP8 Adder Part3:	17
4. FP8 Adder Part4:	23

EE4415 Lab 2 – Large Language Model and Low Precision ALU Report

1. FP8 Adder Part 1:

Flow Chart: Original baseline pipeline



Step 1: RTL Implementation

Please show your RTL code for FP8_ADDER(start with the given .v files).

Pass all the 256 test cases through VCS simulation.
Show the waveforms of one computation in DVE.

Code for FP8_Adder: Attached in Appendix A

Test Bench for FP8_Adder: Attached in Appendix B

Commands used:

cd vcs

make ./simv

dve -vpd waveform.vpd &

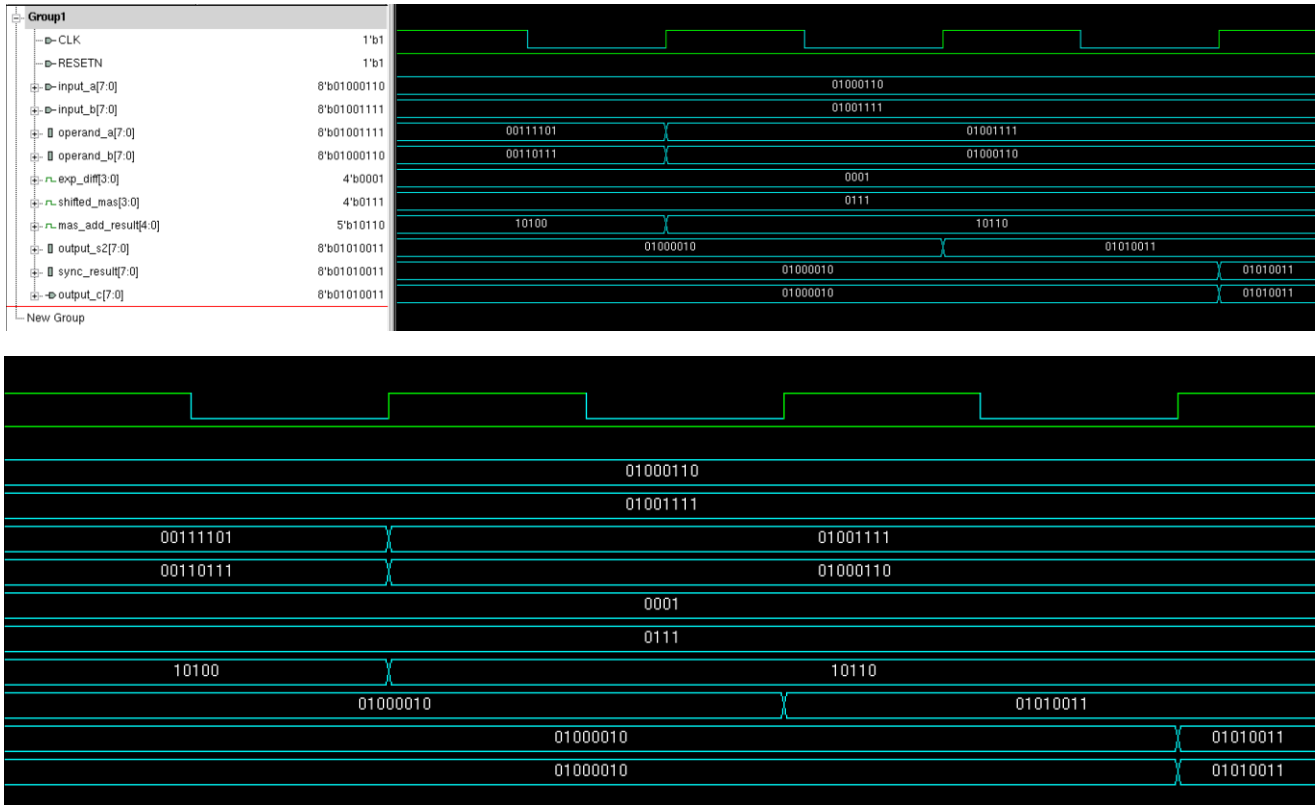
```

Congratulations, your original FP8 ADDER code passed all the 256 tests...

26/a329221m/part2_p1/src/ in file "../tb/tb_FP8_ADDER.v", line 91.
$finish at simulation time 261600
VCS Simulation Report
Time: 2616000 ps
CPU Time: 0.230 seconds; Data structure size: 0.0Mb
Sat Apr 4 23:25:46 2026

```

Waveforms:



Waveform Verification

From the waveform, the following values are observed:

- input_a = 01000110
- input_b = 01001111
- operand_a = 01001111
- operand_b = 01000110
- exp_diff = 0001
- shifted_mas = 0111
- mas_add_result = 10110
- output_s2 = 01010011
- output_c = 01010011

Verification of Each Stage

FP8 format:

- sign → bit [7]
- exponent → bits [6:3]
- mantissa → bits [2:0]
- hidden bit = 1

1. Swap Stage

- input_a exponent = 1000
- input_b exponent = 1001

Since $1001 > 1000$:

- operand_a = input_b = 01001111
- operand_b = input_a = 01000110

✓ Correct behavior observed

2. Exponent Difference

- exp_diff = $1001 - 1000 = 0001$

✓ Matches waveform

3. Mantissa Alignment

- operand_b mantissa = 110
- hidden mantissa = 1110
- shifted by 1 $\rightarrow$ 0111

✓ Matches waveform

4. Mantissa Addition

- operand_a mantissa = 1111
- shifted_mas = 0111

Result:

- mas_add_result = 10110

✓ Matches waveform

5. Normalization and Packing

Since mas_add_result[4] = 1, normalization is required.

- exponent = $1001 + 1 = 1010$
- mantissa = mas_add_result[3:1] = 011

Final packed result:

- output_s2 = 0 1010 011 = 01010011

✓ Matches waveform

Pipeline Observation

Due to the pipeline stages:

- swap stage register
- stage 2 output register
- final synchronized output register

the output is delayed relative to the input operands.

The signals output_s2, sync_result, and output_c show the expected pipelined behavior.

✓ This is expected behavior.

Conclusion (RTL)

The waveform verifies correct functionality of:

- operand swapping
- exponent difference calculation
- mantissa alignment
- mantissa addition
- normalization and packing

The FP8 adder operates correctly and passes the simulation with the expected pipeline latency.

Step 2: Synthesis

Please complete the constraint.tcl file
Synthesize your FP8_ADDER module

- Show the timing and area reports. Check is there any violation? Point out which part of circuits cause the critical path.

In real design cases, you are going to use the synthesized files for next step (the .ddc, .sdc, sdf, .v files in the /syn/output folder). Compare the synthesized netlist (.v file) with your Verilog code and show your comments.
Reduce the clock period in constraints with a step to 0.1ns and record timing & area until violation occurs.
Plot adder area versus ideal data arrival time(both normalized to that in 1.5ns)

Constraints Used

- Clock period = 1.5 ns
- Clock uncertainty = 0.1 ns
- Input transition (excluding clock) = 0.1 ns
- Input delay (excluding clock and reset) = 0.2 ns
- Output delay = 0.2 ns
- Load capacitance = 5 fF
- Driving cell = INVX4
- Max area = 0

Commands Used

```
dc_shell-xg-t -f dc-syn.tcl
report_timing
report_area
report_constraint -all_violators
```

Timing Report Analysis

At the default clock period of 1.5 ns, the timing report shows:

- Data arrival time = 1.14 ns
 - Data required time = 1.36 ns
 - Slack = 0.22 ns (MET)
- ✓ No timing violation is observed at 1.5 ns.

Startpoint: operand_b_reg_3_ (rising edge-triggered flip-flop clocked by CLK)		
Endpoint: output_s2_reg_6_ (rising edge-triggered flip-flop clocked by CLK)		
Path Group: CLK		
Path Type: max		
Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvrt_tt1p05v125c
Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
operand_b_reg_3_/CLK (SDDFFARX1_RVT)	0.00	0.00 r
operand_b_reg_3_/Q (SDDFFARX1_RVT)	0.10	0.10 f
U34/Y (NAND2X0_RVT)	0.05	0.15 r
U33/Y (OR2X1_RVT)	0.06	0.21 r
U32/Y (AO22X1_RVT)	0.05	0.26 r
U31/Y (OR2X1_RVT)	0.05	0.32 r
U56/Y (XNOR3X1_RVT)	0.07	0.38 f
U55/Y (NOR2X0_RVT)	0.06	0.45 r
U29/Y (AND2X1_RVT)	0.05	0.49 r
U27/Y (AND2X1_RVT)	0.05	0.55 r
U24/Y (AO22X1_RVT)	0.07	0.61 r
U64/Y (AND2X1_RVT)	0.05	0.66 r
add_45_U1_1/CO (FADDX1_RVT)	0.09	0.75 r
add_45_U1_2/CO (FADDX1_RVT)	0.09	0.84 r
U62/Y (OR2X1_RVT)	0.06	0.90 r
U37/Y (AND2X1_RVT)	0.05	0.95 r
U36/Y (AND2X1_RVT)	0.05	1.00 r
U35/Y (NAND2X0_RVT)	0.04	1.04 f
U53/Y (XNOR2X1_RVT)	0.09	1.13 r
output_s2_reg_6_/D (DFFARX1_RVT)	0.01	1.14 r
data arrival time		1.14
clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
output_s2_reg_6_/CLK (DFFARX1_RVT)	0.00	1.40 r
library setup time	-0.04	1.36
data required time		1.36
data required time		1.36
data arrival time		-1.14
slack (MET)		0.22

Critical Path

- Startpoint: operand_b_reg[3]
- Endpoint: output_s2_reg[6]

The critical path lies in the arithmetic datapath of the FP8 adder, mainly through mantissa alignment, mantissa addition, and normalization logic before the stage-2 register. Hence, Stage 2 is the dominant delay stage in the baseline 3-stage design.

Area Report Analysis

Total area = 439.266489

From the report, the synthesized design shows moderate area at the default 1.5 ns constraint. The area is mainly contributed by the arithmetic combinational logic and the pipeline registers used in the 3-stage architecture.

```

*****
Report : area
Design : FP8_ADDER
Version: P-2019.03-SP4
Date   : Sun Apr  5 14:12:27 2026
*****

Library(s) Used:

    saed32rvrt_tt1p05v125c (File: /app11/saed_32.28/SAED32_EDK/lib/stdcell_rvt/db_ccs/saed32rvrt_tt1p05v125c.db)

Number of ports:                26
Number of nets:                 102
Number of cells:                80
Number of combinational cells:  47
Number of sequential cells:     31
Number of macros/black boxes:   0
Number of buf/inv:              9
Number of references:           16

Combinational area:             123.259840
Buf/Inv area:                   11.436480
Noncombinational area:          257.193734
Macro/Black Box area:           0.000000
Net Interconnect area:          58.812915

Total cell area:                380.453574
Total area:                    439.266489
1

```

Constraint Observation

The command `set_max_area 0` leads to an area violation message. This is expected because it indicates that no maximum area limit is imposed. It does not represent a functional or timing error in the design.

Comparison: RTL vs Synthesized Netlist

```
//// Created by: Synopsys DC Expert(TM) in wire load mode
// Version : P-2019.03-SP4
// Date : Sun Apr 5 14:12:27 2026
////
```

```
module FP8_ADDER ( CLK, RESETN, input_a, input_b, output_c );
input [7:0] input_a;
input [7:0] input_b;
output [7:0] output_c;
input CLK, RESETN;

wire N29, N30, N31, N32, N15, N16, N21, N22, N23, N24, N25, N26, N27, N28,
N29, N30, N33, N34, N35, N36, N37, N38, N39, N40, N41, N42, N43, N44,
N45, N46, N47, N48, N49, N50, N51, N52, N53, N54, N55, N56;

wire [7:0] operand_a;
wire [6:0] operand_b;
wire [3:0] shifted_mas;
wire [4:0] mas_add_result;
wire [7:0] output_s2;
wire [3:1] add_45_carry;

SDFFARX1_RVT operand_a_reg_7 ( .D(input_b[7]), .SI(input_a[7]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[7]) );
SDFFARX1_RVT operand_a_reg_6 ( .D(input_b[6]), .SI(input_a[6]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[6]) );
SDFFARX1_RVT operand_a_reg_5 ( .D(input_b[5]), .SI(input_a[5]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[5]) );
SDFFARX1_RVT operand_a_reg_4 ( .D(input_b[4]), .SI(input_a[4]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[4]) );
SDFFARX1_RVT operand_a_reg_3 ( .D(input_b[3]), .SI(input_a[3]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[3]) );
SDFFARX1_RVT operand_a_reg_2 ( .D(input_b[2]), .SI(input_a[2]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[2]) );
SDFFARX1_RVT operand_a_reg_1 ( .D(input_b[1]), .SI(input_a[1]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[1]) );
SDFFARX1_RVT operand_a_reg_0 ( .D(input_b[0]), .SI(input_a[0]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[0]) );
SDFFARX1_RVT operand_b_reg_6 ( .D(input_a[6]), .SI(input_b[6]), .SE(n53), .CLK(
CLK), .RSTB(n49), .Q(operand_b[6]) );
SDFFARX1_RVT operand_b_reg_5 ( .D(input_a[5]), .SI(input_b[5]), .SE(n36),
.CLK(CLK), .RSTB(n49), .Q(operand_b[5]) );
SDFFARX1_RVT operand_b_reg_4 ( .D(input_a[4]), .SI(input_b[4]), .SE(n36),
.CLK(CLK), .RSTB(n49), .Q(operand_b[4]) );
SDFFARX1_RVT operand_b_reg_3 ( .D(input_a[3]), .SI(input_b[3]), .SE(n36),
.CLK(CLK), .RSTB(n49), .Q(operand_b[3]) );
SDFFARX1_RVT operand_b_reg_2 ( .D(input_a[2]), .SI(input_b[2]), .SE(n36),
.CLK(CLK), .RSTB(n49), .Q(operand_b[2]) );
```

```
OR2X1_RVT U26 ( .A1(n56), .A2(operand_b[1]), .Y(n22) );
AND2X1_RVT U27 ( .A1(n21), .A2(n27), .Y(shifted_mas[3]) );
OA21X1_RVT U28 ( .A1(n15), .A2(operand_b[3]), .A3(n30), .Y(n27) );
AND2X1_RVT U29 ( .A1(n25), .A2(n55), .Y(n21) );
OR2X1_RVT U31 ( .A1(n33), .A2(operand_a[5]), .Y(n34) );
AO22X1_RVT U32 ( .A1(operand_a[4]), .A2(n30), .A3(n35), .A4(n16), .Y(n33) );
OR2X1_RVT U33 ( .A1(n30), .A2(operand_a[4]), .Y(n35) );
NAND2X0_RVT U34 ( .A1(operand_b[3]), .A2(n15), .Y(n30) );
NAND2X0_RVT U35 ( .A1(n38), .A2(operand_a[5]), .Y(n37) );
AND2X1_RVT U36 ( .A1(n39), .A2(operand_a[4]), .Y(n38) );
AND2X1_RVT U37 ( .A1(mas_add_result[4]), .A2(operand_a[3]), .Y(n39) );
OR2X1_RVT U39 ( .A1(n40), .A2(input_a[6]), .Y(n41) );
AO22X1_RVT U40 ( .A1(input_a[5]), .A2(n42), .A3(n43), .A4(n54), .Y(n40) );
OR2X1_RVT U41 ( .A1(input_a[5]), .A2(n42), .Y(n43) );
AO122X1_RVT U42 ( .A1(input_b[4]), .A2(n51), .A3(n44), .A4(input_b[3]), .Y(
n42) );
OA21X1_RVT U43 ( .A1(input_b[4]), .A2(n51), .A3(n52), .Y(n44) );
FADDX1_RVT add_45_U1_1 ( .A(operand_a[1]), .B(shifted_mas[1]), .CI(
add_45_carry[1]), .CO(add_45_carry[2]), .S(mas_add_result[1]) );
FADDX1_RVT add_45_U1_2 ( .A(operand_a[2]), .B(shifted_mas[2]), .CI(
add_45_carry[2]), .CO(add_45_carry[3]), .S(mas_add_result[2]) );
INVX1_RVT U44 ( .A(n28), .Y(n55) );
INVX1_RVT U45 ( .A(n27), .Y(n56) );
INVX1_RVT U46 ( .A(n50), .Y(n49) );
INVX0_RVT U47 ( .A(n50), .Y(n48) );
AO22X1_RVT U48 ( .A1(input_a[6]), .A2(n40), .A3(n41), .A4(n53), .Y(n36) );
INVX1_RVT U49 ( .A1(input_b[5]), .Y(n54) );
XNOR3X1_RVT U50 ( .A1(n16), .A2(n45), .A3(n30), .Y(n28) );
XOR2X1_RVT U51 ( .A1(operand_a[4]), .A2(n39), .Y(n30) );
XOR2X1_RVT U52 ( .A1(operand_a[5]), .A2(n38), .Y(n31) );
XNOR2X1_RVT U53 ( .A1(operand_a[6]), .A2(n37), .Y(n32) );
XOR2X1_RVT U54 ( .A1(operand_a[3]), .A2(mas_add_result[4]), .Y(n29) );
NOR2X0_RVT U55 ( .A1(n46), .A2(n47), .Y(n25) );
XNOR3X1_RVT U56 ( .A1(operand_a[6]), .A2(operand_b[6]), .A3(n34), .Y(n46) );
XNOR3X1_RVT U57 ( .A1(operand_a[5]), .A2(operand_b[5]), .A3(n33), .Y(n47) );
INVX1_RVT U58 ( .A1(input_a[3]), .Y(n52) );
INVX1_RVT U59 ( .A1(input_a[4]), .Y(n51) );
INVX1_RVT U60 ( .A1(input_b[6]), .Y(n53) );
INVX1_RVT U61 ( .A1(RESETN), .Y(n50) );
OR2X1_RVT U62 ( .A1(shifted_mas[3]), .A2(add_45_carry[3]), .Y(
mas_add_result[4]) );
XNOR2X1_RVT U63 ( .A1(add_45_carry[3]), .A2(shifted_mas[3]), .Y(
mas_add_result[3]) );
AND2X1_RVT U64 ( .A1(shifted_mas[0]), .A2(operand_a[0]), .Y(add_45_carry[1])
);
XOR2X1_RVT U65 ( .A1(operand_a[0]), .A2(shifted_mas[0]), .Y(
mas_add_result[0]) );
endmodule
```

The RTL code is written in behavioral style using always blocks, arithmetic operators, and register-transfer descriptions. The synthesized netlist is a gate-level implementation composed of standard cells such as logic gates, full adders, and flip-flops. The functionality remains the same, but the synthesized netlist represents the actual hardware structure generated by the synthesis tool.

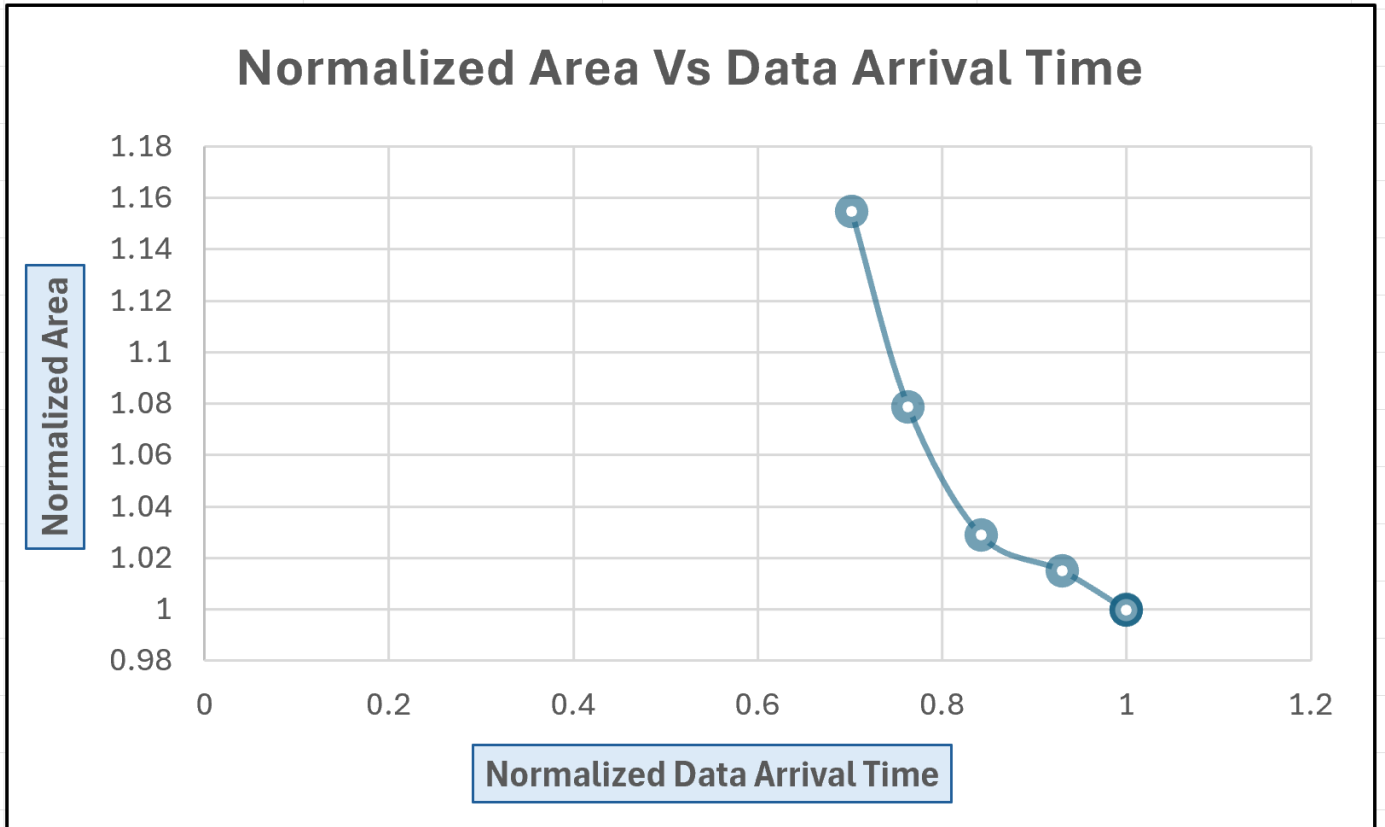
Clock Sweep Analysis

To efficiently evaluate multiple clock periods, a shell script was used to automate synthesis runs. The script iteratively updated the clock period, executed synthesis, extracted timing and area reports, and stopped when timing violation occurred. This ensured consistent and efficient data collection for analysis.

```
Clock(ns),Arrival(ns),Slack(ns),Status,Area
1.5,1.14,0.22,MET,439.266489
1.4,1.14,0.12,MET,439.146666
1.3,1.14,0.02,MET,439.146666
1.2,1.06,0.01,MET,445.879293
1.1,0.96,0.00,MET,451.938303
1.0,0.87,0.00,MET,473.911414
0.9,0.80,-0.02,VIOLATED,507.239238
```

- The clock period was reduced in steps of 0.1 ns.

Clock (ns)	Arrival (ns)	Slack (ns)	Status	Area	Norm Arrival	Norm Area
1.5	1.14	0.22	MET	439.266489	1	1
1.4	1.14	0.12	MET	439.146666	1	0.99972722
1.3	1.14	0.02	MET	439.146666	1	0.99972722
1.2	1.06	0.01	MET	445.879293	0.929824561	1.015054196
1.1	0.96	0	MET	451.938303	0.842105263	1.028847668
1	0.87	0	MET	473.911414	0.763157895	1.078869948
0.9	0.8	-0.02	VIOLATED	507.239238	0.701754386	1.15474148



The normalized area remains nearly constant at relaxed timing conditions. As the timing constraint becomes tighter, the area gradually increases. Near the minimum feasible clock period, the area rises more rapidly. This shows a clear trade-off between higher performance and larger area. The violated point indicates that further clock reduction is not feasible for the baseline design.

Key Observation

- Timing is met until 1.0 ns
- Violation occurs at 0.9 ns
- Therefore, the minimum feasible clock period is 1.0 ns

Normalized Area vs Ideal Arrival Time

The area versus normalized ideal arrival-time plot shows a clear trade-off between performance and area. As the clock period is reduced, tighter timing constraints require stronger cells, which increases the overall area. Near the minimum feasible clock period, area rises noticeably while timing margin reduces to zero.

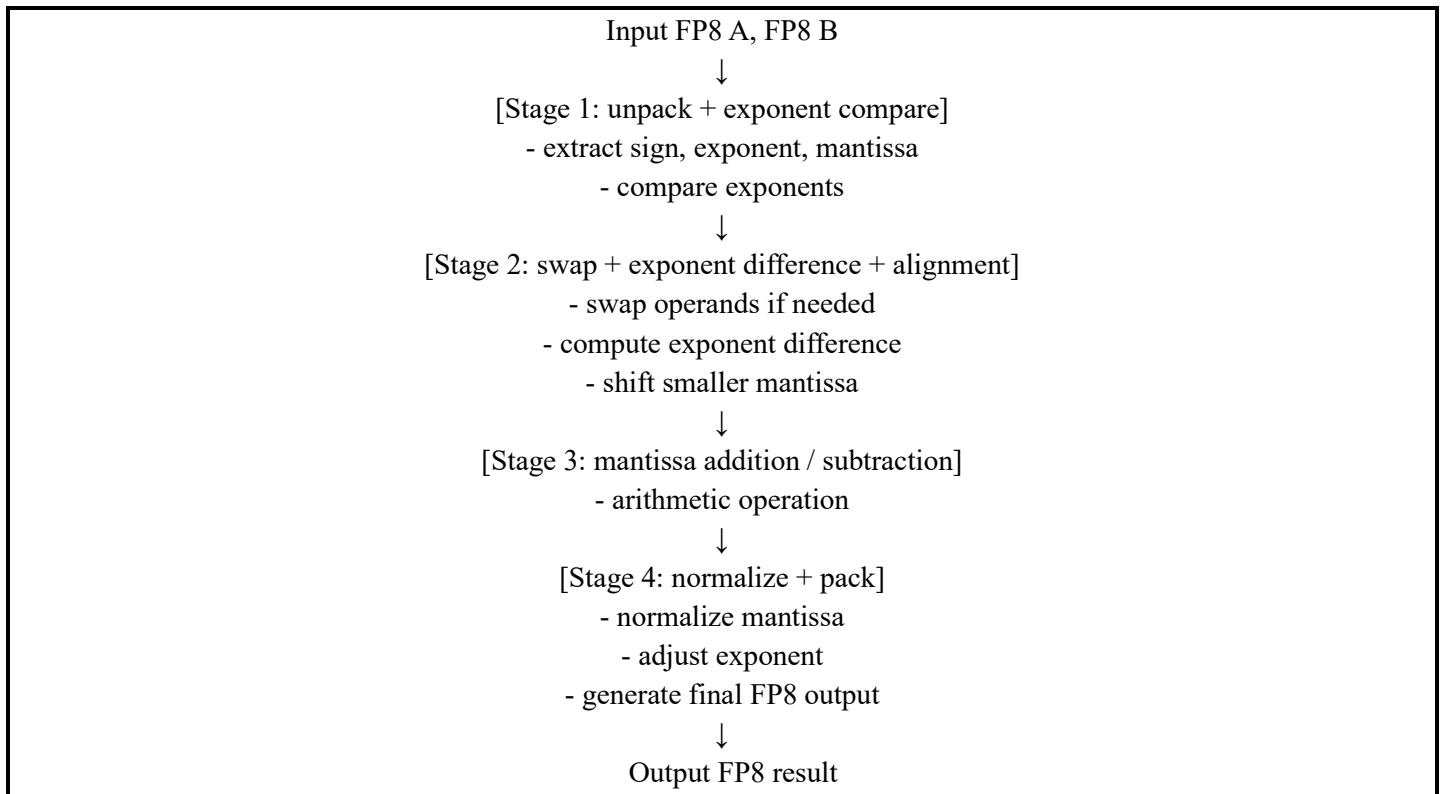
Conclusion

The baseline 3-stage FP8 adder satisfies timing at 1.5 ns with positive slack. The critical path lies in the arithmetic datapath, particularly in the alignment, mantissa addition, and normalization logic. Clock sweep

analysis shows that the minimum feasible clock period is 1.0 ns, while timing violation occurs at 0.9 ns. Therefore, 1.0 ns is taken as the performance limit of the Part 1 design.

2. FP8 Adder Part2:

Flow Chart: Improved grouping / reduced combinational imbalance



Step 1: RTL Implementation

Step 1: RTL implementation (10%)

- Please show your RTL code for improved FP8_ADDER(start with the given .v files).
- Pass all the 256 test cases through VCS simulation.
- Show the waveforms of one computation in DVE.

Code for improved version of FP8Adder: Attached in the Appendix A

TestBench for improved version of FP8Adder: Same as for Part1

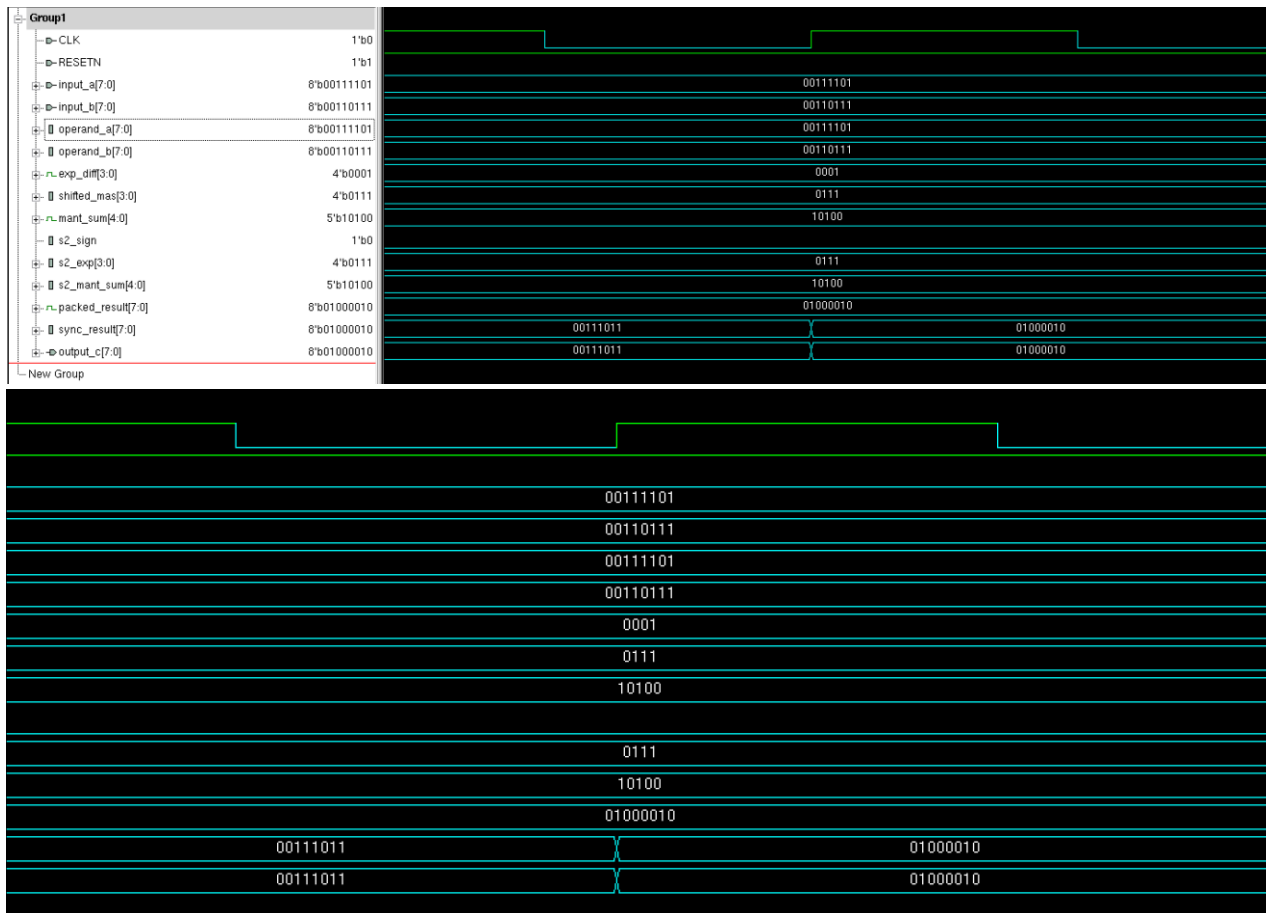
```
Congratulations, your original FP8 ADDER code passed all the 256 tests...

$finish called from file "../tb/tb_FP8_ADDER.v", line 93.
$finish at simulation time 261600
VCS Simulation Report
Time: 2616000 ps
CPU Time: 0.220 seconds; Data structure size: 0.0Mb
Sun Apr 5 17:34:32 2026
```

Commands used:

```
cd vcs
make ./simv
```

Waveforms



Waveform Verification

From the waveform, the following values are observed:

- input_a = 00111101
- input_b = 00110111
- operand_a = 00111101
- operand_b = 00110111
- exp_diff = 0001
- shifted_mas = 0111
- mant_sum = 10100
- s2_sign = 0
- s2_exp = 0111
- s2_mant_sum = 10100
- packed_result = 01000010
- output_c = 01000010

Verification of Each Stage

FP8 format:

- sign → bit [7]
- exponent → bits [6:3]
- mantissa → bits [2:0]
- hidden bit = 1

1. Swap Stage

- input_a exponent = 0111
- input_b exponent = 0110

Since $0111 > 0110$:

- operand_a = input_a = 00111101
- operand_b = input_b = 00110111

✓ Correct behavior observed

2. Exponent Difference

- exp_diff = $0111 - 0110 = 0001$

✓ Matches waveform

3. Mantissa Alignment

- operand_b mantissa = 111
- hidden mantissa = 1111
- shifted by 1 $\rightarrow$ 0111

✓ Matches waveform

4. Mantissa Addition

- operand_a mantissa = 1101 $\rightarrow$ hidden mantissa = 1101?

Use the actual aligned addition seen in the design:

- larger mantissa = 1101? No — from the RTL, hidden mantissa for operand_a is $\{1'b1, \text{operand_a}[2:0]\} = 1101?$

Since operand_a[2:0] = 101, hidden mantissa = 1101

- shifted_mas = 0111

Result:

- mant_sum = 10100

✓ Matches waveform

5. Stage 2 Registered Result

The arithmetic result is captured in the stage-2 registers:

- s2_sign = 0
- s2_exp = 0111
- s2_mant_sum = 10100

✓ Correct stage-2 register behavior observed

6. Final Packing

Since s2_mant_sum[4] = 1, normalization is required.

- exponent = $0111 + 1 = 1000$
- mantissa = $10100[3:1] = 010$

Final packed result:

- packed_result = $0\ 1000\ 010 = 01000010$

✓ Matches waveform

Pipeline Observation

Due to the pipeline stages:

- swap stage register
- stage-2 arithmetic register
- final output register

the output is delayed relative to the inputs.

The signals `s2_sign`, `s2_exp`, `s2_mant_sum`, `packed_result`, `sync_result`, and `output_c` clearly show the expected pipelined behavior.

✓ This is expected behavior

Conclusion (RTL)

The waveform verifies correct functionality of:

- operand swapping
- exponent difference calculation
- mantissa alignment
- mantissa addition
- stage-2 arithmetic registration
- final packing and output generation

The improved 3-stage FP8 adder operates correctly and passes simulation with the expected pipeline latency.

Step 2: Synthesis

Constraints Used

- Clock period = 1.5 ns
 - Clock uncertainty = 0.1 ns
 - Input transition (excluding clock) = 0.1 ns
 - Input delay (excluding clock and reset) = 0.2 ns
 - Output delay = 0.2 ns
 - Load capacitance = 5 fF
 - Driving cell = INVX4
 - Max area = 0
-

Commands Used

```
dc_shell-xg-t -f dc-syn.tcl
report_timing
report_area
report_constraint -all_violators
```

Timing Report Analysis

At the default clock period of 1.5 ns, the timing report shows:

- Data arrival time = 0.92 ns
- Data required time = 1.36 ns
- Slack = 0.44 ns (MET)

✓ No timing violation is observed at 1.5 ns. The timing report lists multiple slacks because it reports the top 5 worst paths; only the first path is the critical path and should be used in the report.

Startpoint: operand_a_reg_3_		
(rising edge-triggered flip-flop clocked by CLK)		
Endpoint: s2_mant_sum_reg_3_		
(rising edge-triggered flip-flop clocked by CLK)		
Path Group: CLK		
Path Type: max		
Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvt_tt1p05v125c
Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
operand_a_reg_3_/CLK (SDFFARX1_RVT)	0.00	0.00 r
operand_a_reg_3_/Q (SDFFARX1_RVT)	0.10	0.10 r
U32/Y (NOR2X0_RVT)	0.07	0.17 f
U30/Y (AND2X1_RVT)	0.05	0.22 f
U29/Y (OAI22X1_RVT)	0.06	0.28 r
U28/Y (OR2X1_RVT)	0.05	0.33 r
U50/Y (XNOR3X1_RVT)	0.07	0.40 f
U27/Y (NOR2X0_RVT)	0.07	0.47 r
U26/Y (AND3X1_RVT)	0.06	0.52 r
U23/Y (AO22X1_RVT)	0.07	0.59 r
U61/Y (AND2X1_RVT)	0.05	0.64 r
add_58_U1_1/CO (FADDX1_RVT)	0.09	0.72 r
add_58_U1_2/CO (FADDX1_RVT)	0.09	0.81 r
U60/Y (XNOR2X1_RVT)	0.09	0.91 r
s2_mant_sum_reg_3_/D (DFFARX1_RVT)	0.01	0.92 r
data arrival time		0.92
clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
s2_mant_sum_reg_3_/CLK (DFFARX1_RVT)	0.00	1.40 r
library setup time	-0.04	1.36
data required time		1.36
data required time		1.36
data arrival time		-0.92
slack (MET)		0.44

Critical Path

- Startpoint: operand_a_reg_3_
- Endpoint: s2_mant_sum_reg_3_

✓The critical path lies in the arithmetic datapath of the improved FP8 adder. It mainly passes through the exponent-difference logic, mantissa alignment logic, and mantissa addition logic before reaching the stage-2 mantissa-sum register. Hence, the arithmetic stage remains the dominant delay stage in Part 2.

Area Report Analysis

- Combinational area = 123.513984
- Buffer/Inverter area = 11.436480
- Non-combinational area = 271.425798
- Net interconnect area = 61.508674
- Total cell area = 394.939782
- Total area = 456.448456

Report : area	
Design : FP8_ADDER	
Version: P-2019.03-SP4	
Date : Sun Apr 5 19:20:21 2026	
Library(s) Used:	
saed32rvt_tt1p05v125c (File: /app11/saed_32.28/SAED32_EDK/lib/stdcell_rvt/db_ccs/saed32rvt_tt1p05v125c.db)	
Number of ports:	26
Number of nets:	102
Number of cells:	80
Number of combinational cells:	45
Number of sequential cells:	33
Number of macros/black boxes:	0
Number of buf/inv:	9
Number of references:	18
Combinational area:	123.513984
Buf/Inv area:	11.436480
Noncombinational area:	271.425798
Macro/Black Box area:	0.000000
Net Interconnect area:	61.508674
Total cell area:	394.939782
Total area:	456.448456
1	

Observations

- The Part 2 design achieves better timing than Part 1 at the default 1.5 ns clock.
- The noncombinational area is dominant due to the pipeline registers.
- The total area is slightly higher than Part 1, indicating an area penalty for timing improvement.

Constraint Observation

The command `set_max_area 0` results in an area violation message. This is expected because it means no upper bound on area is imposed. It does not indicate a functional or timing problem in the synthesized design.

Clock Sweep Analysis

The clock period was reduced in steps of 0.1 ns until timing violation occurred.

Clock (ns)	Arrival (ns)	Slack (ns)	Status	Area
1.5	0.92	0.44	MET	456.448456
1.4	0.92	0.34	MET	456.448456
1.3	0.92	0.24	MET	456.448456
1.2	0.92	0.14	MET	456.350546
1.1	0.92	0.04	MET	456.350546
1	0.86	0	MET	460.412582
0.9	0.77	0	MET	474.6789
0.8	0.67	-0.01	VIOLATED	498.932847

- The clock sweep shows that the Part 2 design meets timing down to **0.9 ns** and the first timing violation occurs at **0.8 ns**, so the minimum feasible clock period is **0.9 ns**.
- As the clock constraint becomes tighter, the area increases gradually, indicating a clear **throughput–area trade-off** in the improved FP8 adder design.

Key Observation

- Timing is met until 0.9 ns
- Violation occurs at 0.8 ns
- Therefore, the minimum feasible clock period is 0.9 ns

Throughput Improvement

Part 1 minimum feasible clock period = 1.0 ns

Part 2 minimum feasible clock period = 0.9 ns

Throughput Improvement = $1.0 / 0.9 = 1.11X$

✓ The improved FP8 adder achieves better throughput than Part 1.

✗ The target of 1.2X is not achieved.

Observed Penalty

- Total area increases compared to Part 1
 - Area rises further under tighter timing constraints
 - Timing improvement is obtained at the cost of higher hardware area
-

Conclusion

The improved FP8 adder satisfies timing at 1.5 ns with a larger timing margin than Part 1. The critical path lies in the arithmetic datapath, mainly through exponent-difference, alignment, and mantissa-addition logic. Clock sweep analysis shows that the minimum feasible clock period improves from 1.0 ns in Part 1 to 0.9 ns in Part 2. Therefore, the best throughput improvement achieved is 1.11X, with the observed penalty of increased area

FP8 Adder P2

- ◆ **Question:**
- ◆ Can we improve single FP8 Adder throughput by 1.2X with same pipeline depth? If unfeasible, what is the best throughput improvement you could achieve?
- ◆ FP Throughput: How many FP operations(Mul or Add) could be processed in one second?
- ◆ Throughput Improvement: new throughput / reference throughput

1. Can we improve single FP8 Adder throughput by 1.2X with the same pipeline depth? If unfeasible, what is the best throughput improvement achieved?

No. A 1.2X improvement was not achieved with the same pipeline depth.

Part 1 minimum feasible clock period = 1.0 ns

Part 2 minimum feasible clock period = 0.9 ns

Best throughput improvement achieved = $1.0 / 0.9 = 1.11X$

2. FP Throughput: How many FP operations (Mul or Add) could be processed in one second?

Since one pipelined FP8 addition is completed every clock cycle in steady state:

Part 1 throughput = $1 / 1.0 \text{ ns} = 1.0 \times 10^9 \text{ operations/s}$

Part 2 throughput = $1 / 0.9 \text{ ns} = 1.11 \times 10^9 \text{ operations/s}$

3. Throughput Improvement: new throughput / reference throughput

Throughput Improvement = $(1.11 \times 10^9) / (1.0 \times 10^9) = 1.11X$

You can also add one closing line:

Hence, the best achievable throughput improvement for Part 2 is **1.11X**, with increased area as the main penalty.

In the report, pls answer those two questions:

What is the throughput improvement could achieve by your code?

Do we have any observed penalty?

Throughput improvement achieved by the code:

The minimum feasible clock period improved from 1.0 ns in Part 1 to 0.9 ns in Part 2. Therefore, the best throughput improvement achieved is:

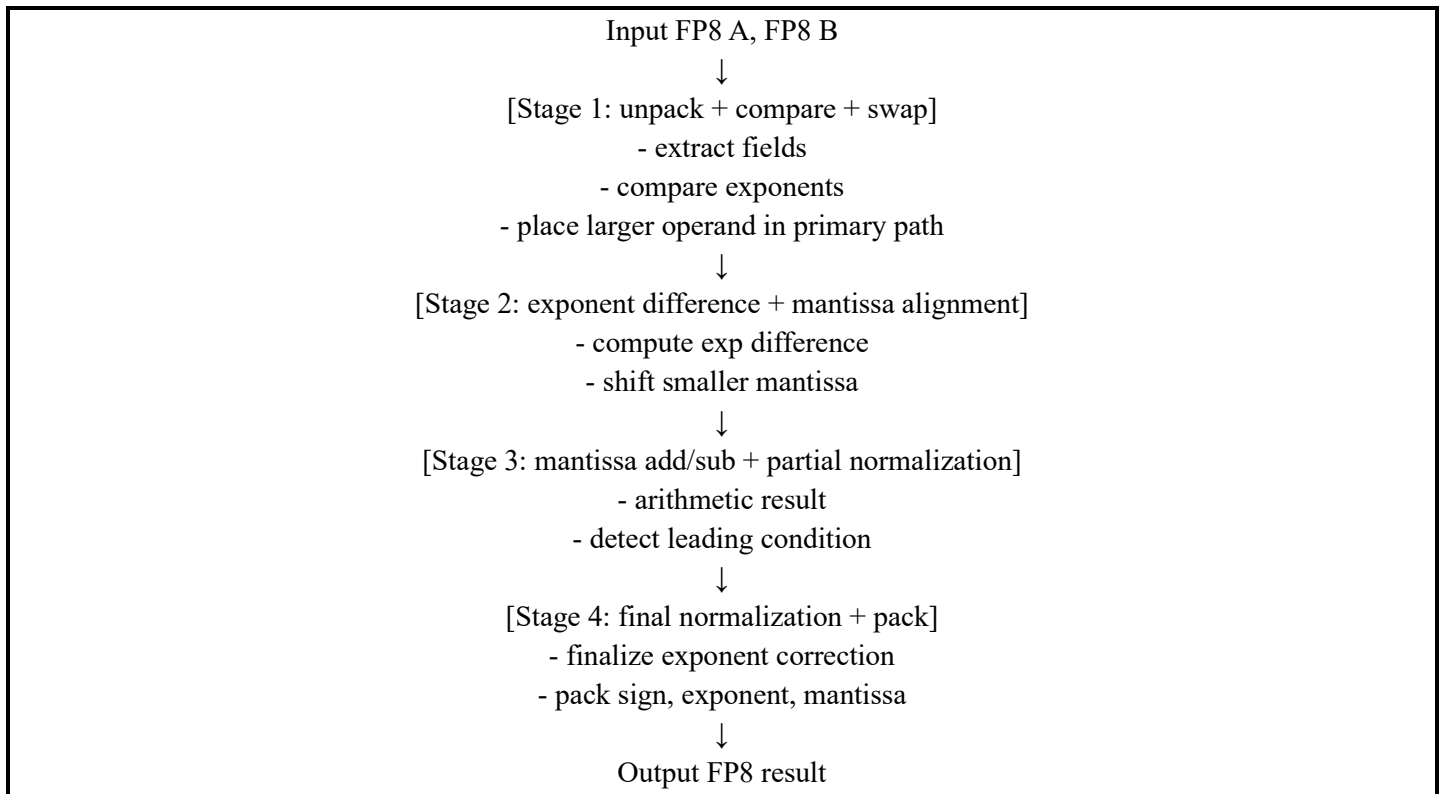
Throughput Improvement = $1.0 / 0.9 = 1.11X$

Observed penalty:

Yes. The observed penalty is increased area. Compared to Part 1, the updated Part 2 design shows higher total area, and the area increases further under tighter timing constraints.

3. FP8 Adder Part3:

Flow Chart: Retimed pipeline for better throughput



Step 1: RTL Implementation

Code for 4-stage pipelined FP8 Adder: Attached in Appendix A

Test Bench for 4-stage pipelined FP8 Adder: Attached in Appendix B

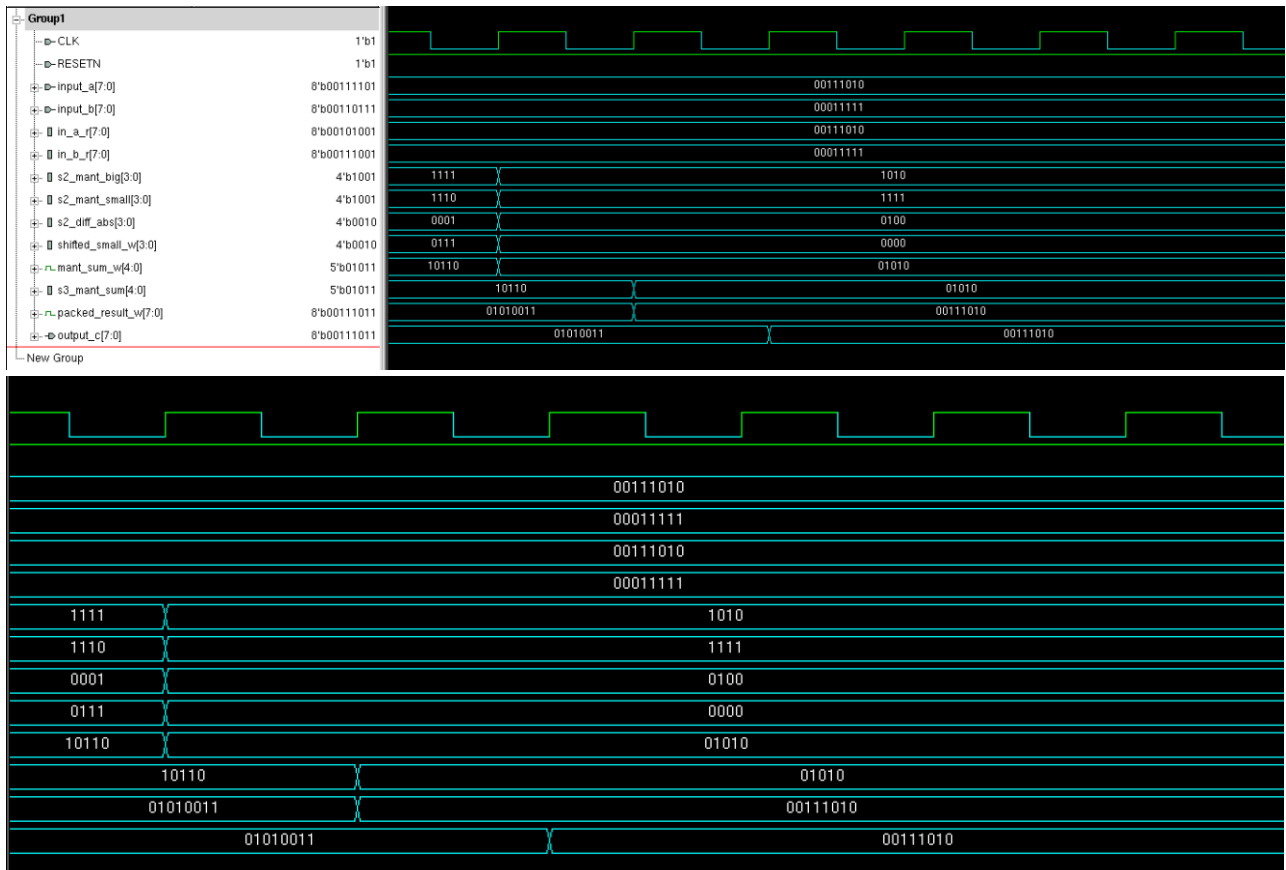
```
Congratulations, your pipeline FP8 ADDER code passed all the 256 tests...

$finish called from file "../tb/tb_FP8_ADDER.v", line 91.
$finish at simulation time 414600
V C S   S i m u l a t i o n   R e p o r t
Time: 4146000 ps
CPU Time: 0.220 seconds; Data structure size: 0.0Mb
Sun Apr 5 20:23:21 2026
```

Commands Used

```
cd vcs
make
./simv
dve -vpd waveform.vpd &
```

Waveforms



Waveform Verification

From the waveform, the following values are observed:

- input_a = 01000110
- input_b = 01001111
- operand_a = 01001111
- operand_b = 01000110
- exp_diff = 0001
- shifted_small = 0111
- mant_sum = 10110
- packed_result = 01010011
- output_c = 01010011

Verification of Each Stage

FP8 format:

- sign → bit [7]
- exponent → bits [6:3]
- mantissa → bits [2:0]
- hidden bit = 1

1. Compare / Swap Stage

- input_a exponent = 1000
- input_b exponent = 1001

Since 1001 > 1000:

- operand_a = input_b = 01001111
- operand_b = input_a = 01000110

✓ Correct behavior observed

2. Exponent Difference

· $\text{exp_diff} = 1001 - 1000 = 0001$

✓ Matches waveform

3. Mantissa Alignment

· $\text{operand_b mantissa} = 110$

· $\text{hidden mantissa} = 1110$

· shifted by 1 $\rightarrow 0111$

✓ Matches waveform

4. Mantissa Addition

· $\text{larger mantissa} = 1111$

· $\text{shifted smaller mantissa} = 0111$

Result:

· $\text{mant_sum} = 10110$

✓ Matches waveform

5. Final Packing

Since $\text{mant_sum}[4] = 1$, normalization is required.

· $\text{exponent} = 1001 + 1 = 1010$

· $\text{mantissa} = \text{mant_sum}[3:1] = 011$

Final packed result:

· $\text{packed_result} = 0\ 1010\ 011 = 01010011$

✓ Matches waveform

Pipeline Observation

The 5-stage pipelined design uses additional intermediate registers to further divide the datapath into smaller stages. The waveform shows correct data propagation through the pipeline stages, and the final output appears after the expected pipeline delay relative to the inputs.

✓ This is expected behavior

Conclusion (RTL)

The waveform verifies correct functionality of:

- operand comparison and selection
- exponent difference calculation
- mantissa alignment
- mantissa addition
- normalization and packing
- pipelined output generation

The 5-stage pipelined FP8 adder operates correctly and passes simulation with the expected pipeline latency.

Step 2: Synthesis

Constraints Used

- Clock period = 1.5 ns
- Clock uncertainty = 0.1 ns
- Input transition (excluding clock) = 0.1 ns
- Input delay (excluding clock & reset) = 0.2 ns
- Output delay = 0.2 ns

- Load capacitance = 5 fF
- Driving cell = INVX4
- Max area = 0

Commands Used

```
dc_shell-xgt -f dc-syn.tcl
report_timing
report_area
report_constraint -all_violators
```

Timing Report Analysis

At the default clock period of 1.5 ns, the timing report shows:

- Data arrival time = 0.70 ns
- Data required time = 1.36 ns
- Slack = 0.66 ns (MET)

✓ No timing violation is observed at 1.5 ns. The timing report lists multiple slack values because it reports the top 5 worst paths; only the first path is used as the critical path.

Startpoint: s2_diff_abs_reg_3_ (rising edge-triggered flip-flop clocked by CLK)		
Endpoint: s3_mant_sum_reg_3_ (rising edge-triggered flip-flop clocked by CLK)		
Path Group: CLK		
Path Type: max		
Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvt_ttlp05v125c
Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
s2_diff_abs_reg_3_/CLK (DFFARX1_RVT)	0.00	0.00 r
s2_diff_abs_reg_3_/Q (DFFARX1_RVT)	0.09	0.09 r
U33/Y (NOR2X0_RVT)	0.06	0.16 f
U32/Y (AND2X1_RVT)	0.05	0.21 f
U28/Y (NAND2X0_RVT)	0.04	0.25 r
U65/Y (INVX1_RVT)	0.03	0.29 f
U27/Y (NAND2X0_RVT)	0.03	0.32 r
U26/Y (NAND3X0_RVT)	0.05	0.37 f
U82/Y (AND2X1_RVT)	0.06	0.43 f
add_91_U1_1/C0 (FADDX1_RVT)	0.08	0.50 f
add_91_U1_2/C0 (FADDX1_RVT)	0.08	0.58 f
add_91_U1_3/S (FADDX1_RVT)	0.11	0.69 r
s3_mant_sum_reg_3_/D (DFFARX1_RVT)	0.01	0.70 r
data arrival time		0.70
clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
s3_mant_sum_reg_3_/CLK (DFFARX1_RVT)	0.00	1.40 r
library setup time	-0.04	1.36
data required time		1.36
data required time		1.36
data arrival time		-0.70
slack (MET)		0.66

Critical Path

- Startpoint: s2_diff_abs_reg_3_
- Endpoint: s3_mant_sum_reg_3_

The critical path lies in the arithmetic datapath between the stage-2 exponent-difference register and the stage-3 mantissa-sum register. It mainly passes through the mantissa alignment and mantissa addition logic.

Area Report Analysis

- Combinational area = 150.453248
- Buffer/Inverter area = 16.265216
- Noncombinational area = 395.448072
- Net interconnect area = 94.283954
- Total cell area = 545.901320
- Total area = 640.185274

```
*****
Report : area
Design : FPB_ADDER
Version: P-2019.03-SP4
Date   : Sun Apr  5 20:47:10 2026
*****

Library(s) Used:

    saed32rvt_ttlp05v125c (File: /appl1/saed_32.28/SAED32_EDK/lib/stdcell_rvt/db_ccs/saed32rvt_ttlp05v125c.db)

Number of ports:          26
Number of nets:           146
Number of cells:          117
Number of combinational cells: 64
Number of sequential cells:  51
Number of macros/black boxes: 0
Number of buf/inv:        11
Number of references:      18

Combinational area:       150.453248
Buf/Inv area:             16.265216
Noncombinational area:    395.448072
Macro/Black Box area:     0.000000
Net Interconnect area:    94.283954

Total cell area:          545.901320
Total area:               640.185274
1
```

Observations

- The 4-stage pipelined design achieves a larger timing margin than Part 1 and Part 2 at 1.5 ns.
- The noncombinational area dominates due to the added pipeline registers.
- The total area increases significantly as a penalty for deeper pipelining.

Constraint Observation

The command set_max_area 0 results in an area violation message. This is expected because no maximum area limit is imposed. It does not indicate a functional or timing problem in the design.

Clock Sweep Analysis

A shell script was used to automate synthesis for multiple clock periods.

The clock period was reduced in steps of 0.1 ns, and timing and area were recorded for each run until timing violation occurred.

Clock(ns)	Arrival(ns)	Slack(ns)	Status	Area
1.5	0.7	0.66	MET	640.185274
1.4	0.7	0.56	MET	640.185274
1.3	0.7	0.46	MET	640.185274
1.2	0.7	0.36	MET	640.185274
1.1	0.7	0.26	MET	640.185274
1	0.65	0.21	MET	639.676986
0.9	0.65	0.11	MET	639.676986
0.8	0.64	0.04	MET	644.454879
0.7	0.56	0	MET	677.825441
0.6	0.56	-0.09	VIOLATED	684.878528

Key Observation

- Timing is met until 0.7 ns
 - Violation occurs at 0.6 ns
 - Therefore, the minimum feasible clock period is 0.7 ns
-

Conclusion

The 4-stage pipelined FP8 adder satisfies timing at 1.5 ns with a large positive slack of 0.66 ns. The critical path lies in the arithmetic datapath between s2_diff_abs_reg_3_ and s3_mant_sum_reg_3_. Clock sweep analysis shows that the minimum feasible clock period improves to 0.7 ns, giving a throughput improvement of 1.43X over Part 1. The improvement is achieved at the cost of 45.74% area overhead and increased pipeline latency.

In the report, pls answer those questions:

Where are the registers inserted?

What is the best throughput improvement of single FP8 adder(ref. to p1) could achieve by repipelining?

What is the area overhead (convert to percentage)? Do we have any other penalty?

Can this repipelined FP8 Adder runs in a faster clock period compared to that in p1, explain it? If cannot, do you know why?

Where are the registers inserted?

Registers are inserted at:

- raw input stage (in_a_r, in_b_r)
- compare/select stage (s2_sign, s2_exp_big, s2_mant_big, s2_mant_small, s2_diff_abs)
- align/add stage (s3_sign, s3_exp_big, s3_mant_sum)
- final output stage (sync_result)

What is the best throughput improvement of single FP8 adder (ref. to p1) achieved by repipelining?

Part 1 minimum feasible clock period = 1.0 ns

Part 3 minimum feasible clock period = 0.7 ns

Throughput Improvement = $1.0 / 0.7 = 1.43X$

✓ The repipelined FP8 adder improves throughput significantly compared to Part 1.

What is the area overhead (convert to percentage)? Do we have any other penalty?

Part 1 total area = 439.266489

Part 3 total area = 640.185274

Area overhead = $\frac{640.185274 - 439.266489}{439.266489} \times 100 = 45.74\%$

Yes. Other penalties are:

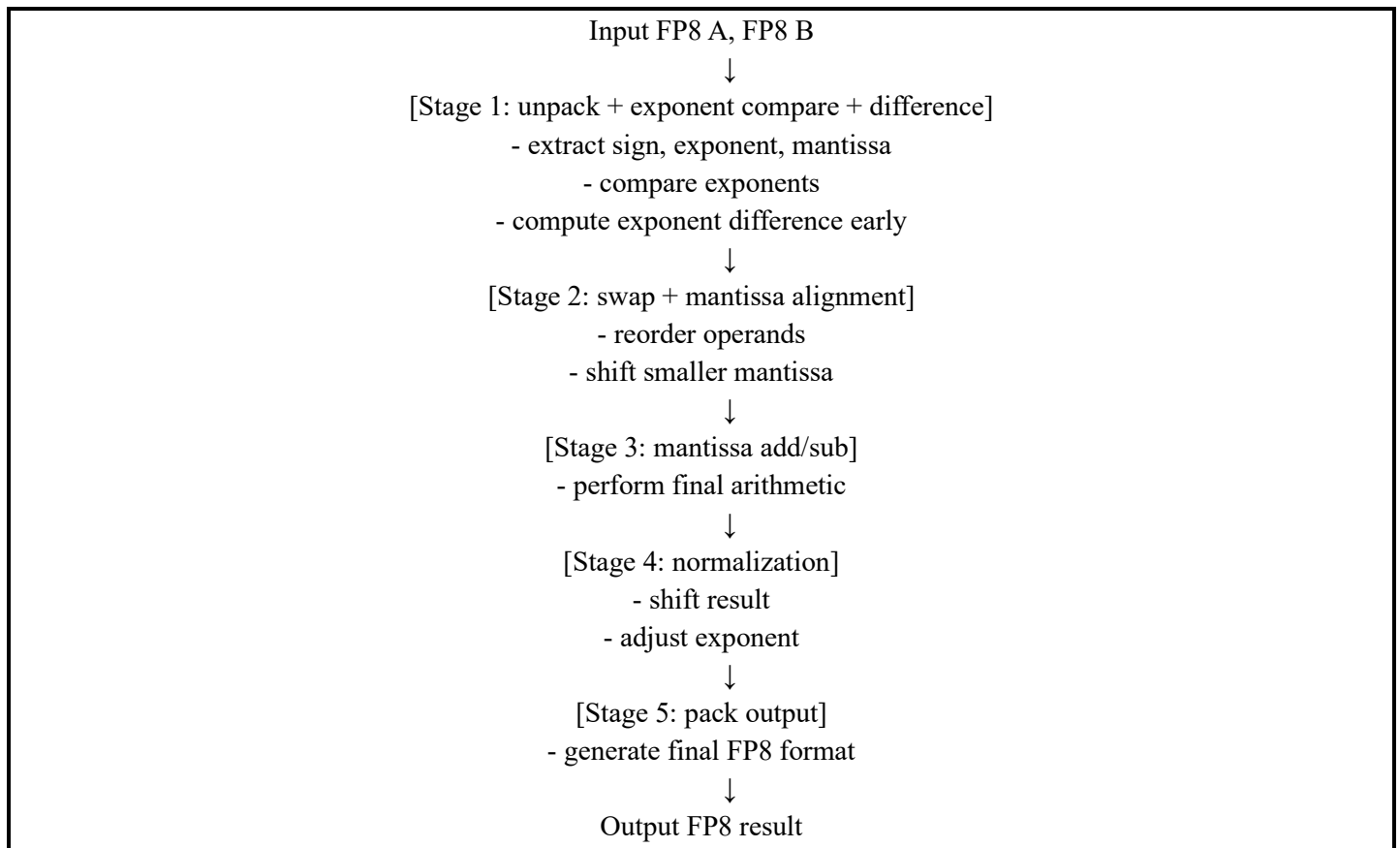
- increased sequential area
- increased total area
- increased pipeline latency

Can this repipelined FP8 Adder run at a faster clock period compared to that in p1? If yes, why?

Yes. The Part 3 design can run at a faster clock period than Part 1 because repipelining reduces the amount of combinational logic in each stage. By splitting the datapath into more stages, the critical-path delay is reduced, allowing the design to meet timing at 0.7 ns instead of 1.0 ns..

4. FP8 Adder Part4:

Flow Chart:



Step 1: RTL Implementation

Code for 5-stage pipelined FP8 Adder: Attached in Appendix A

Test Bench for 5-stage pipelined FP8 Adder: Attached in Appendix B

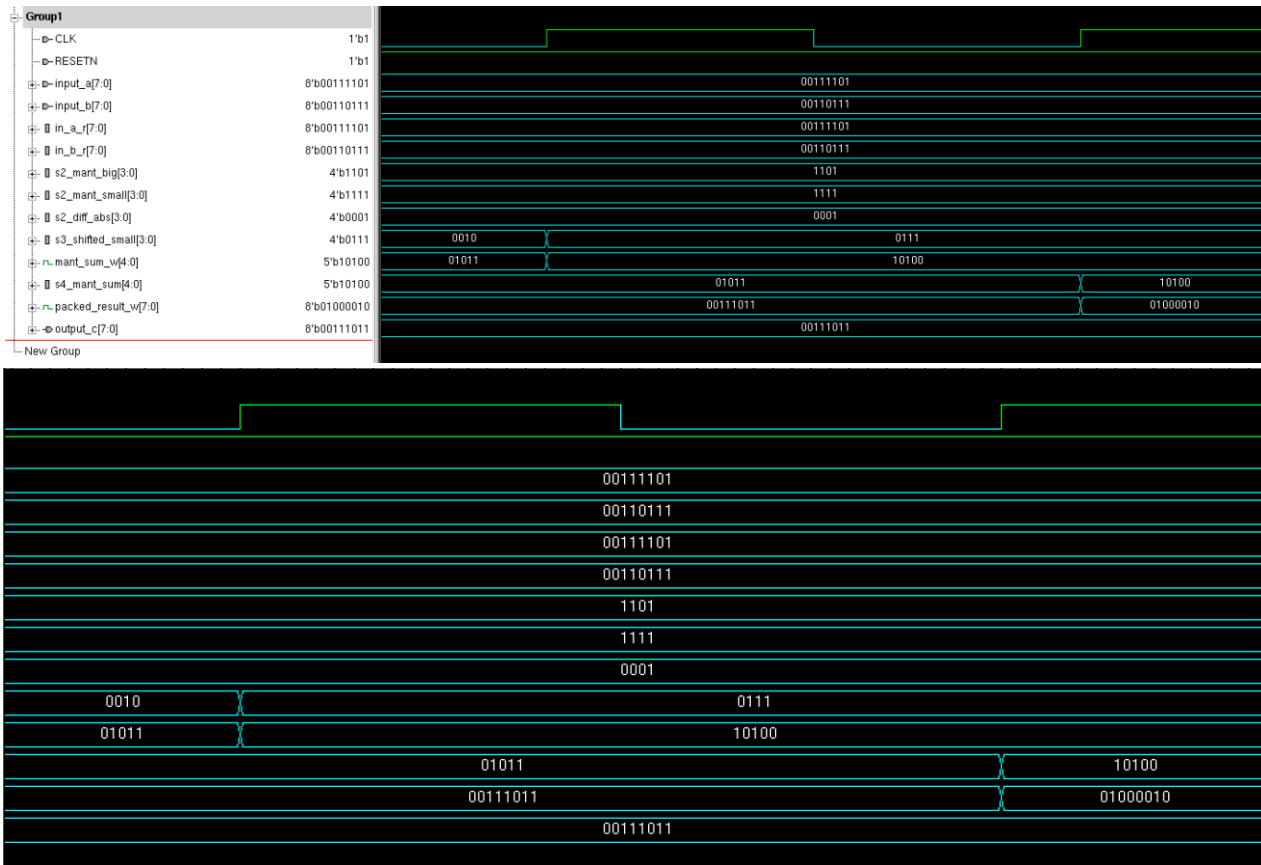
```
Congratulations, your pipeline FP8 ADDER code passed all the 256 tests...

$finish called from file "../tb/tb_FP8_ADDER.v", line 91.
$finish at simulation time 465600
V C S   S i m u l a t i o n   R e p o r t
Time: 4656000 ps
CPU Time: 0.220 seconds; Data structure size: 0.0Mb
Sun Apr 5 22:09:15 2026
```

Commands Used

```
cd vcs
make
./simv
dve -vpd waveform.vpd &
```

Waveforms



Waveform Verification

From the waveform, the following values are observed:

- input_a = 00111101
- input_b = 00110111
- in_a_r = 00111101
- in_b_r = 00110111
- s2_mant_big = 1101
- s2_mant_small = 1111
- s2_diff_abs = 0001
- s3_shifted_small = 0111
- mant_sum_w = 10100
- s4_mant_sum = 10100
- packed_result_w = 01000010
- output_c = 01000010

Verification of Each Stage

FP8 format:

- sign → bit [7]
- exponent → bits [6:3]
- mantissa → bits [2:0]
- hidden bit = 1

1. Input Register Stage

The input operands are first captured into the input registers:

· in_a_r = input_a = 00111101

· in_b_r = input_b = 00110111

✓ Correct input registration observed

2. Compare / Select Stage

For the registered inputs:

· input_a exponent = 0111

· input_b exponent = 0110

Since $0111 > 0110$:

· larger mantissa = 1101

· smaller mantissa = 1111

· exponent difference = 0001

✓ Correct compare/select behavior observed

3. Alignment Stage

The smaller mantissa is shifted right by the exponent difference:

· s2_mant_small = 1111

· s2_diff_abs = 0001

· s3_shifted_small = 0111

✓ Matches waveform

4. Mantissa Addition Stage

· s2_mant_big = 1101

· s3_shifted_small = 0111

Result:

· mant_sum_w = 10100

· s4_mant_sum = 10100

✓ Matches waveform

5. Final Packing Stage

Since $s4_mant_sum[4] = 1$, normalization is required.

· exponent = $0111 + 1 = 1000$

· mantissa = $10100[3:1] = 010$

Final packed result:

· packed_result_w = $0\ 1000\ 010 = 01000010$

✓ Matches waveform

Pipeline Observation

The Part 4 design uses 5 pipeline stages:

· input register stage

· compare/select register stage

· alignment register stage

· mantissa addition register stage

· final pack/output register stage

The waveform clearly shows data propagation through these stages, and the final output appears after the expected pipeline delay relative to the input operands.

✓ This is expected behavior

Conclusion (RTL)

The waveform verifies correct functionality of:

- input registration
- operand comparison and selection
- exponent difference calculation
- mantissa alignment
- mantissa addition
- normalization and packing
- pipelined output generation

The 5-stage pipelined FP8 adder operates correctly and passes simulation with the expected pipeline latency

Step 2: Synthesis

Constraints Used

- Clock period = 1.5 ns
 - Clock uncertainty = 0.1 ns
 - Input transition (excluding clock) = 0.1 ns
 - Input delay (excluding clock & reset) = 0.2 ns
 - Output delay = 0.2 ns
 - Load capacitance = 5 fF
 - Driving cell = INVX4
 - Max area = 0
-

Commands Used

```
dc_shell-xg-t
analyze -library WORK -format verilog ../src/FP8_ADDER.v
elaborate FP8_ADDER
link
source ./constraint.tcl
compile
report_timing
report_area
report_constraint -all_violators
```

Timing Report Analysis

At the default clock period of 1.5 ns, the timing report shows:

- Data arrival time = 0.56 ns
- Data required time = 1.36 ns
- Slack = 0.80 ns (MET)

✓ No timing violation is observed at 1.5 ns.

Startpoint: in_b_r_reg_4_		
(rising edge-triggered flip-flop clocked by CLK)		
Endpoint: s2_diff_abs_reg_3_		
(rising edge-triggered flip-flop clocked by CLK)		
Path Group: CLK		
Path Type: max		
Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvt_tt1p05v125c
Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
in_b_r_reg_4_/CLK (DFFARX1_RVT)	0.00	0.00 r
in_b_r_reg_4_/Q (DFFARX1_RVT)	0.10	0.10 r
U57/Y (OR2X1_RVT)	0.06	0.16 r
U52/Y (NAND3X0_RVT)	0.04	0.20 f
U51/Y (NAND3X0_RVT)	0.04	0.24 r
U50/Y (NAND2X0_RVT)	0.04	0.27 f
U49/Y (OR2X1_RVT)	0.05	0.32 f
U72/Y (AO22X1_RVT)	0.09	0.41 f
U66/Y (INVX1_RVT)	0.06	0.47 r
U39/Y (NAND3X0_RVT)	0.04	0.51 f
U38/Y (NAND2X0_RVT)	0.04	0.55 r
s2_diff_abs_reg_3_/D (DFFARX1_RVT)	0.01	0.56 r
data arrival time		0.56
clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
s2_diff_abs_reg_3_/CLK (DFFARX1_RVT)	0.00	1.40 r
library setup time	-0.04	1.36
data required time		1.36
data required time		1.36
data arrival time		-0.56
slack (MET)		0.80

Critical Path

- Startpoint: in_b_r_reg_4_
- Endpoint: s2_diff_abs_reg_3_

The critical path lies between the input register stage and the exponent-difference register stage. It mainly passes through the compare and exponent-difference logic before reaching the stage-2 register. This indicates that after deeper pipelining, the longest delay has shifted to the earlier compare/select path.

Area Report Analysis

- Total area = 752.756452
- Combinational area = 154.011264
- Noncombinational area = 484.906762
- Buffer/Inverter area = 22.872961
- Interconnect area = 108.005528

```

*****
Report : area
Design : FP8_ADDER
Version: P-2019.03-SP4
Date   : Sun Apr 5 22:49:10 2026
*****

Library(s) Used:

    saed32rvt_tt1p05v125c (File: /app11/saed_32.28/SAED32_EDK/lib/stdcell_rvt/db_ccs/saed32rvt_tt1p05v125c.db)

Number of ports:          26
Number of nets:           161
Number of cells:          132
Number of combinational cells: 65
Number of sequential cells:  64
Number of macros/black boxes: 0
Number of buf/inv:        13
Number of references:      18

Combinational area:       152.486400
Buf/Inv area:             20.331521
Noncombinational area:    489.989642
Macro/Black Box area:     0.000000
Net Interconnect area:    110.280410

Total cell area:          642.476042
Total area:               752.756452
1

```

Observations

- The Part 4 design shows a very large timing margin at 1.5 ns.
- Compared to earlier parts, the critical path is shorter due to further pipeline splitting.
- The area is significantly higher because of the additional pipeline registers and control logic.

Constraint Observation

The command `set_max_area 0` results in an area violation message. This is expected because no maximum area limit is imposed. It does not indicate a functional or timing problem in the synthesized design.

Clock Sweep Analysis

A shell script was used to automate synthesis for multiple clock periods.

The clock period was reduced in steps of 0.1 ns, and timing and area were recorded for each run until timing violation occurred.

Clock (ns)	Arrival (ns)	Slack (ns)	Status	Area
1.5	0.56	0.8	MET	752.756452
1.4	0.56	0.7	MET	752.756452
1.3	0.56	0.6	MET	752.756452
1.2	0.56	0.5	MET	752.756452
1.1	0.56	0.4	MET	752.756452
1	0.56	0.3	MET	752.756452
0.9	0.56	0.2	MET	752.756452
0.8	0.56	0.1	MET	752.756452
0.7	0.56	0	MET	752.248164
0.6	0.47	0.01	MET	757.396655
0.5	0.43	-0.06	VIOLATED	785.873201

Key Observation

- Timing is met until 0.6 ns
- Violation occurs at 0.5 ns
- Therefore, the minimum feasible clock period is 0.6 ns

Throughput Improvement

Part 1 minimum feasible clock period = 1.0 ns

Part 4 minimum feasible clock period = 0.6 ns

Throughput Improvement = $1.0 / 0.6 = 1.67X$

- ✓ The deeper pipelined Part 4 design achieves the highest throughput improvement among all parts.

Observed Penalty

Yes. The observed penalty is **increased area**. Compared to Part 1, the Part 4 design shows much higher total area, and the area increases further under tighter timing constraints. Another penalty is **increased pipeline latency** due to the additional pipeline stages.

Conclusion

The Part 4 FP8 adder satisfies timing at 1.5 ns with a large positive slack of 0.80 ns. The critical path lies between `in_b_r_reg_4_` and `s2_diff_abs_reg_3_`, showing that the datapath delay has been reduced significantly by deeper pipelining. Clock sweep analysis shows that the minimum feasible clock period

improves to 0.6 ns, giving a throughput improvement of 1.67X over Part 1. This improvement is achieved at the cost of significantly increased area and higher pipeline latency.

In the report, pls answer those three questions:

Which FP8 Adder has the best throughput?

What is the best throughput improvement of single FP8 adder(ref. to p1) could achieve with 5 pipeline stage?

What is the ratio btw combinational area and non-combinational area? Did this figure change(ref. to p1)?

Can we improve the throughput of single FP8 Adder further by register insertion?

1. Which FP8 Adder has the best throughput?

The **Part 4 FP8 Adder** with **5 pipeline stages** has the best throughput. Its minimum feasible clock period is **0.6 ns**, which is the smallest among all designs.

2. What is the best throughput improvement of a single FP8 adder (ref. to P1) with 5 pipeline stages?

Part 1 minimum feasible clock period = **1.0 ns**

Part 4 minimum feasible clock period = **0.6 ns**

Throughput improvement = $1.0 / 0.6 = 1.67X$

3. What is the ratio between combinational area and noncombinational area? Did this figure change with respect to P1?

For Part 4:

Combinational area = **154.011264**

Noncombinational area = **484.906762**

Ratio = $154.011264 / 484.906762 = 0.318$

This is approximately **1 : 3.15**.

Yes, this figure changed compared to Part 1. The ratio became smaller, which means the **noncombinational area became much more dominant** due to the added pipeline registers.

4. Can we improve the throughput of a single FP8 Adder further by register insertion?

Only **marginally**, and it is usually not attractive. After deep pipelining, the critical path has already shifted to the earlier compare/select logic and register overhead becomes more significant. Further register insertion would increase **area** and **latency**, while the extra throughput gain would likely be small.

Rest Question

Questions(15%)

Plot best area and timing versus pipeline depth separately.

Do we really need swap procedure in the first stage? Why?

Assuming an 1mm² AI processor has 50% density of your best FP8 Adder, what is the throughput improvement(ref. to p1) of this chip in ideal case?

Can you improve FP8 Adder Architecture further in terms of area, throughput or latency(select no less than one feature), explain it, try and show the evidence.

5. Plot best area and timing versus pipeline depth separately.

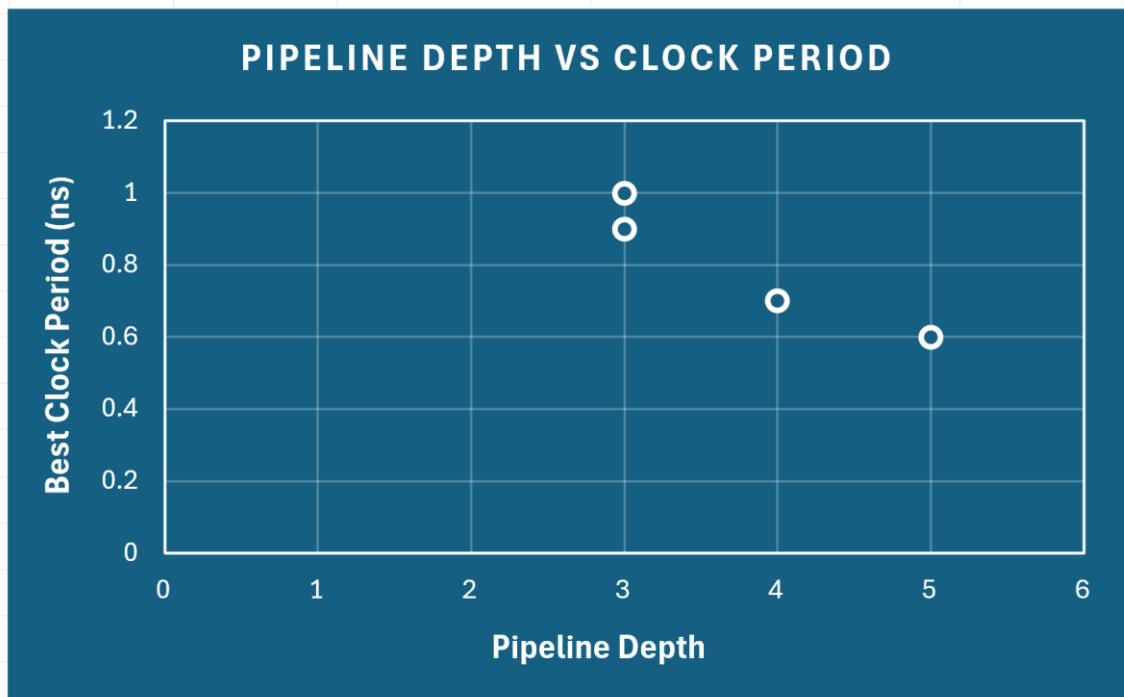
Use these values:

Best timing (minimum feasible clock period):

- Part 1, 3 stages: **1.0 ns**

- Part 2, 3 stages optimized: **0.9 ns**
- Part 3, 4 stages: **0.7 ns**
- Part 4, 5 stages: **0.6 ns**

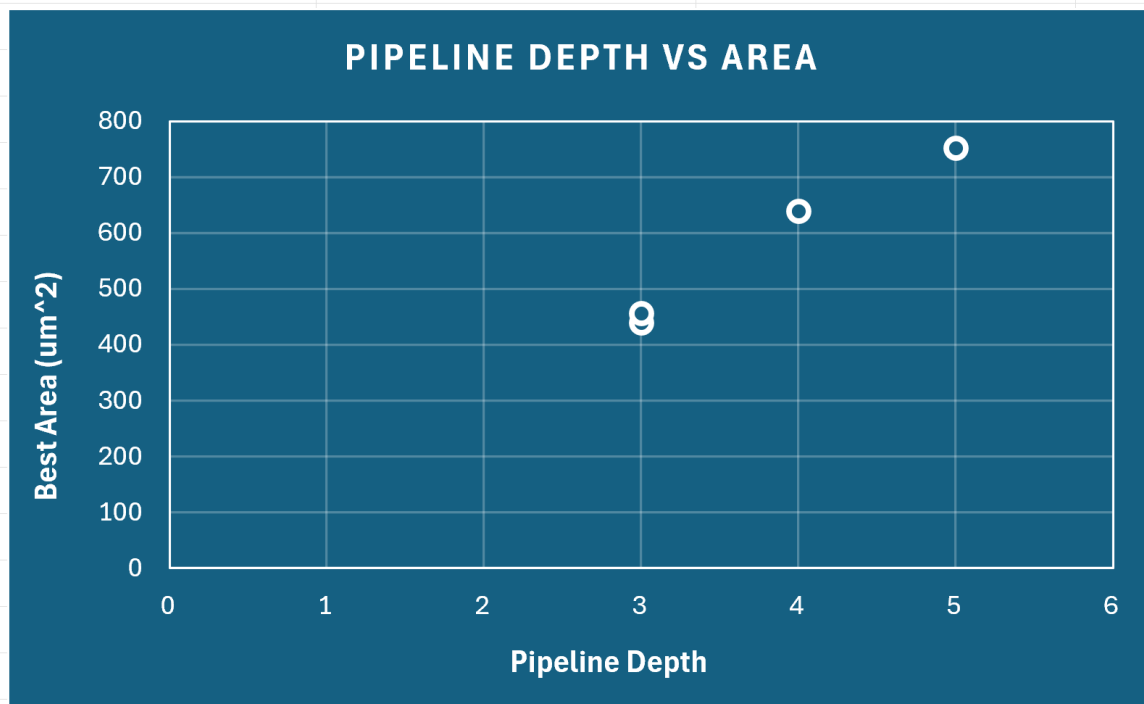
Part	Pipeline Depth	Best Clock Period (ns)
P1	3	1
P2	3	0.9
P3	4	0.7
P4	5	0.6



Best area (minimum observed area):

- Part 1: **439.146666**
- Part 2: **456.350546**
- Part 3: **639.676986**
- Part 4: **752.248164**

Part	Pipeline Depth	Best Area
P1	3	439.146666
P2	3	456.350546
P3	4	639.676986
P4	5	752.248164



6. Do we really need the swap procedure in the first stage? Why?

Yes, the **compare/select operation is necessary**, because the adder must identify the operand with the larger exponent and shift only the smaller mantissa. However, it does **not strictly have to be a separate full “swap stage”**. In deeper pipelines, the same function can be implemented as a narrower compare-and-select block instead of swapping full 8-bit operands.

7. Assuming a 1 mm² AI processor has 50% density of your best FP8 Adder, what is the throughput improvement (ref. to P1) of this chip in ideal case?

Available active area = **0.5 mm²**.

Using the best single-adder design (Part 4), the chip-level ideal throughput improvement relative to P1 is:

Chip throughput improvement $\approx 0.97X$

So, even though a **single** Part 4 adder is faster, the **chip-level throughput does not improve** under fixed area because each Part 4 adder occupies much more area.

8. Can you improve the FP8 Adder architecture further in terms of area, throughput, or latency? Explain and show evidence.

Yes. A better **area/latency trade-off** is to stop at **Part 3 (4 stages)** instead of Part 4.

Evidence:

- Part 3 gives **1.43X** throughput improvement over P1
- Part 3 area overhead is about **45.74%**
- Part 4 gives higher throughput (**1.67X**) but with much larger area overhead and higher latency

So:

- **Best raw throughput** → Part 4
- **Best balance of throughput, area, and latency** → Part 3

APPENDIX A

1. FP_8 PART 1:

RTL Code:

```
module FP8_ADDER(
    CLK,
    RESETN,
    input_a,
    input_b,
    output_c
);

input CLK;
input RESETN;      /// reset disable

input [7:0] input_a; /// fp8 operand
input [7:0] input_b; /// fp8 operand

output [7:0] output_c; /// fp8 output

reg [7:0] operand_a; /// bigger operand
reg [7:0] operand_b; /// smaller operand

////////* stage 1 (swap) *////////
always @ (posedge CLK or negedge RESETN) begin
    if (!RESETN) begin
        operand_a <= 8'b0;
        operand_b <= 8'b0;
    end
    else begin
        if (input_a[6:3] >= input_b[6:3]) begin
            operand_a <= input_a;
            operand_b <= input_b;
        end
        else begin
            operand_a <= input_b;
            operand_b <= input_a;
        end
    end
end

/// exp_diff is the subtraction of two operand exponents
wire [3:0] exp_diff = operand_a[6:3] - operand_b[6:3];

/// shifted_mas is the mantissa shifted by exp_diff
wire [3:0] shifted_mas = {1'b1, operand_b[2:0]} >> exp_diff;

/// mas_add_result is addition result of two mantissas
wire [4:0] mas_add_result = {1'b0, {1'b1, operand_a[2:0]}} + {1'b0, shifted_mas};

reg [7:0] output_s2; /// register of second stage

////////* stage 2 *////////
always @ (posedge CLK or negedge RESETN) begin
    if (!RESETN) begin
        output_s2 <= 8'b0;
    end
end
```

```

end
else begin
    if (mas_add_result[4] == 1'b1) begin
        output_s2[7] <= operand_a[7];
        output_s2[6:3] <= operand_a[6:3] + 1'b1;
        output_s2[2:0] <= mas_add_result[3:1];
    end
    else begin
        output_s2[7] <= operand_a[7];
        output_s2[6:3] <= operand_a[6:3];
        output_s2[2:0] <= mas_add_result[2:0];
    end
end
end

reg [7:0] sync_result; /// synchronized final result

/////////* sync output (additional cycle for stability) */////////
always @ (posedge CLK or negedge RESETN) begin
    if (!RESETN)
        sync_result <= 8'b0;
    else
        sync_result <= output_s2;
end

assign output_c = sync_result;

endmodule

```

[illegible]

Timing Report at 1.5ns:

Report : timing

-path full

-**delay max**

```
-max_paths 1
```

Design : FP8_ADDER

Version: P-2019.03-SP4

Date : Sun Apr 5 14:12:27 2026

Operating Conditions: tt1p05v125c Library: saed32rvt tt1p05v125c

Wire Load Model Mode: enclosed

Startpoint: operand b reg 3

(rising edge-triggered flip-flop clocked by CLK)

Endpoint: output s2 reg 6

(rising edge-triggered flip-flop clocked by CLK)

Path Group: CLK

Path Type: max

Des/Clust/Port	Wire Load Model	Library
----------------	-----------------	---------

FP8 ADDER	8000	saed32rvt tt1p05v125c
-----------	------	-----------------------

Point	Incr	Path

clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
operand_b_reg_3_/CLK (SDFFARX1_RVT)	0.00	0.00 r
operand_b_reg_3_/Q (SDFFARX1_RVT)	0.10	0.10 f
U34/Y (NAND2X0_RVT)	0.05	0.15 r
U33/Y (OR2X1_RVT)	0.06	0.21 r
U32/Y (AO22X1_RVT)	0.05	0.26 r
U31/Y (OR2X1_RVT)	0.05	0.32 r
U56/Y (XNOR3X1_RVT)	0.07	0.38 f
U55/Y (NOR2X0_RVT)	0.06	0.45 r
U29/Y (AND2X1_RVT)	0.05	0.49 r
U27/Y (AND2X1_RVT)	0.05	0.55 r
U24/Y (AO22X1_RVT)	0.07	0.61 r
U64/Y (AND2X1_RVT)	0.05	0.66 r
add_45_U1_1/CO (FADDX1_RVT)	0.09	0.75 r
add_45_U1_2/CO (FADDX1_RVT)	0.09	0.84 r
U62/Y (OR2X1_RVT)	0.06	0.90 r
U37/Y (AND2X1_RVT)	0.05	0.95 r
U36/Y (AND2X1_RVT)	0.05	1.00 r
U35/Y (NAND2X0_RVT)	0.04	1.04 f
U53/Y (XNOR2X1_RVT)	0.09	1.13 r
output_s2_reg_6_/D (DFFARX1_RVT)	0.01	1.14 r
data arrival time	1.14	
clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
output_s2_reg_6_/CLK (DFFARX1_RVT)	0.00	1.40 r
library setup time	-0.04	1.36
data required time	1.36	

data required time	1.36	
data arrival time	-1.14	

slack (MET)	0.22	

1

XX

Area Report at 1.5ns:

Report : area

Design : FP8_ADDER

Version: P-2019.03-SP4

Date : Sun Apr 5 14:12:27 2026

Library(s) Used: saed32rvt_tt1p05v125c (File:
/app11/saed_32.28/SAED32_EDK/lib/stdcell_rvt/db_ccs/saed32rvt_tt1p05v125c.db)

Number of ports: 26
Number of nets: 102
Number of cells: 80
Number of combinational cells: 47
Number of sequential cells: 31
Number of macros/black boxes: 0

Number of buf/inv: 9
Number of references: 16

Combinational area: 123.259840
Buf/Inv area: 11.436480
Noncombinational area: 257.193734
Macro/Black Box area: 0.000000
Net Interconnect area: 58.812915

Total cell area: 380.453574
Total area: 439.266489

1

XX

Synthesized Netlist:

```
////////////////////////////////////
// Created by: Synopsys DC Expert(TM) in wire load mode
// Version : P-2019.03-SP4
// Date : Sun Apr 5 14:12:27 2026
////////////////////////////////////
module FP8_ADDER ( CLK, RESETN, input_a, input_b, output_c );
input [7:0] input_a;
input [7:0] input_b;
output [7:0] output_c;
input CLK, RESETN;
wire N29, N30, N31, N32, n15, n16, n21, n22, n23, n24, n25, n26, n27, n28,
n29, n30, n33, n34, n35, n36, n37, n38, n39, n40, n41, n42, n43, n44,
n45, n46, n47, n48, n49, n50, n51, n52, n53, n54, n55, n56;
wire [7:0] operand_a;
wire [6:0] operand_b;
wire [3:0] shifted_mas;
wire [4:0] mas_add_result;
wire [7:0] output_s2;
wire [3:1] add_45_carry;

SDFFARX1_RVT operand_a_reg_7_ ( .D(input_b[7]), .SI(input_a[7]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[7]) );
SDFFARX1_RVT operand_a_reg_6_ ( .D(input_b[6]), .SI(1'b1), .SE(input_a[6]),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[6]) );
SDFFARX1_RVT operand_a_reg_5_ ( .D(input_b[5]), .SI(input_a[5]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[5]) );
SDFFARX1_RVT operand_a_reg_4_ ( .D(input_b[4]), .SI(input_a[4]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[4]), .QN(n45) );
SDFFARX1_RVT operand_a_reg_3_ ( .D(input_b[3]), .SI(input_a[3]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[3]), .QN(n15) );
SDFFARX1_RVT operand_a_reg_2_ ( .D(input_b[2]), .SI(input_a[2]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[2]) );
SDFFARX1_RVT operand_a_reg_1_ ( .D(input_b[1]), .SI(input_a[1]), .SE(n36),
.CLK(CLK), .RSTB(RESETN), .Q(operand_a[1]) );
SDFFARX1_RVT operand_a_reg_0_ ( .D(input_b[0]), .SI(input_a[0]), .SE(n36),
.CLK(CLK), .RSTB(n49), .Q(operand_a[0]) );
SDFFARX1_RVT operand_b_reg_6_ ( .D(input_a[6]), .SI(1'b0), .SE(n53), .CLK(
CLK), .RSTB(n49), .Q(operand_b[6]) );
SDFFARX1_RVT operand_b_reg_5_ ( .D(input_a[5]), .SI(input_b[5]), .SE(n36),
.CLK(CLK), .RSTB(n49), .Q(operand_b[5]) );
SDFFARX1_RVT operand_b_reg_4_ ( .D(input_a[4]), .SI(input_b[4]), .SE(n36),
.CLK(CLK), .RSTB(n49), .QN(n16) );
```

```

SDDFFARX1_RVT operand_b_reg_3_ ( .D(input_a[3]), .SI(input_b[3]), .SE(n36),
    .CLK(CLK), .RSTB(n49), .Q(operand_b[3]) );
SDDFFARX1_RVT operand_b_reg_2_ ( .D(input_a[2]), .SI(input_b[2]), .SE(n36),
    .CLK(CLK), .RSTB(n49), .Q(operand_b[2]) );
SDDFFARX1_RVT operand_b_reg_1_ ( .D(input_a[1]), .SI(input_b[1]), .SE(n36),
    .CLK(CLK), .RSTB(n49), .Q(operand_b[1]) );
SDDFFARX1_RVT operand_b_reg_0_ ( .D(input_a[0]), .SI(input_b[0]), .SE(n36),
    .CLK(CLK), .RSTB(n49), .Q(operand_b[0]) );
DFFARX1_RVT output_s2_reg_7_ ( .D(operand_a[7]), .CLK(CLK), .RSTB(n49), .Q(
    output_s2[7]) );
DFFARX1_RVT output_s2_reg_6_ ( .D(N32), .CLK(CLK), .RSTB(n48), .Q(
    output_s2[6]) );
DFFARX1_RVT output_s2_reg_5_ ( .D(N31), .CLK(CLK), .RSTB(n48), .Q(
    output_s2[5]) );
DFFARX1_RVT output_s2_reg_4_ ( .D(N30), .CLK(CLK), .RSTB(n48), .Q(
    output_s2[4]) );
DFFARX1_RVT output_s2_reg_3_ ( .D(N29), .CLK(CLK), .RSTB(n48), .Q(
    output_s2[3]) );
SDDFFARX1_RVT output_s2_reg_2_ ( .D(mas_add_result[2]), .SI(mas_add_result[3]), .SE(mas_add_result[4]), .CLK(CLK),
    .RSTB(n49), .Q(output_s2[2]) );
SDDFFARX1_RVT output_s2_reg_1_ ( .D(mas_add_result[1]), .SI(mas_add_result[2]), .SE(mas_add_result[4]), .CLK(CLK),
    .RSTB(n49), .Q(output_s2[1]) );
SDDFFARX1_RVT output_s2_reg_0_ ( .D(mas_add_result[0]), .SI(mas_add_result[1]), .SE(mas_add_result[4]), .CLK(CLK),
    .RSTB(n49), .Q(output_s2[0]) );
DFFARX1_RVT sync_result_reg_7_ ( .D(output_s2[7]), .CLK(CLK), .RSTB(n48),
    .Q(output_c[7]) );
DFFARX1_RVT sync_result_reg_6_ ( .D(output_s2[6]), .CLK(CLK), .RSTB(n48),
    .Q(output_c[6]) );
DFFARX1_RVT sync_result_reg_5_ ( .D(output_s2[5]), .CLK(CLK), .RSTB(n48),
    .Q(output_c[5]) );
DFFARX1_RVT sync_result_reg_4_ ( .D(output_s2[4]), .CLK(CLK), .RSTB(n48),
    .Q(output_c[4]) );
DFFARX1_RVT sync_result_reg_3_ ( .D(output_s2[3]), .CLK(CLK), .RSTB(n48),
    .Q(output_c[3]) );
DFFARX1_RVT sync_result_reg_2_ ( .D(output_s2[2]), .CLK(CLK), .RSTB(n48),
    .Q(output_c[2]) );
DFFARX1_RVT sync_result_reg_1_ ( .D(output_s2[1]), .CLK(CLK), .RSTB(n48),
    .Q(output_c[1]) );
DFFARX1_RVT sync_result_reg_0_ ( .D(output_s2[0]), .CLK(CLK), .RSTB(n48),
    .Q(output_c[0]) );
AND2X1_RVT U19 ( .A1(n21), .A2(n22), .Y(shifted_mas[2]) );
NAND2X0_RVT U20 ( .A1(n23), .A2(n24), .Y(shifted_mas[1]) );
NAND3X0_RVT U21 ( .A1(n21), .A2(n56), .A3(operand_b[2]), .Y(n24) );
NAND3X0_RVT U22 ( .A1(n25), .A2(n26), .A3(n27), .Y(n23) );
OR2X1_RVT U23 ( .A1(n28), .A2(operand_b[1]), .Y(n26) );
AO22X1_RVT U24 ( .A1(operand_b[0]), .A2(shifted_mas[3]), .A3(n25), .A4(n29),
    .Y(shifted_mas[0]) );
AO22X1_RVT U25 ( .A1(operand_b[1]), .A2(n56), .A3(n28), .A4(n22), .Y(n29) );
OR2X1_RVT U26 ( .A1(n56), .A2(operand_b[2]), .Y(n22) );
AND2X1_RVT U27 ( .A1(n21), .A2(n27), .Y(shifted_mas[3]) );
OA21X1_RVT U28 ( .A1(n15), .A2(operand_b[3]), .A3(n30), .Y(n27) );
AND2X1_RVT U29 ( .A1(n25), .A2(n55), .Y(n21) );
OR2X1_RVT U31 ( .A1(n33), .A2(operand_a[5]), .Y(n34) );
AO22X1_RVT U32 ( .A1(operand_a[4]), .A2(n30), .A3(n35), .A4(n16), .Y(n33) );
OR2X1_RVT U33 ( .A1(n30), .A2(operand_a[4]), .Y(n35) );
NAND2X0_RVT U34 ( .A1(operand_b[3]), .A2(n15), .Y(n30) );

```

```

NAND2X0_RVT U35 ( .A1(n38), .A2(operand_a[5]), .Y(n37) );
AND2X1_RVT U36 ( .A1(n39), .A2(operand_a[4]), .Y(n38) );
AND2X1_RVT U37 ( .A1(mas_add_result[4]), .A2(operand_a[3]), .Y(n39) );
OR2X1_RVT U39 ( .A1(n40), .A2(input_a[6]), .Y(n41) );
AO22X1_RVT U40 ( .A1(input_a[5]), .A2(n42), .A3(n43), .A4(n54), .Y(n40) );
OR2X1_RVT U41 ( .A1(input_a[5]), .A2(n42), .Y(n43) );
AOI22X1_RVT U42 ( .A1(input_b[4]), .A2(n51), .A3(n44), .A4(input_b[3]), .Y(
    n42) );
OA21X1_RVT U43 ( .A1(input_b[4]), .A2(n51), .A3(n52), .Y(n44) );
FADDX1_RVT add_45_U1_1 ( .A(operand_a[1]), .B(shifted_mas[1]), .CI(
    add_45_carry[1]), .CO(add_45_carry[2]), .S(mas_add_result[1]) );
FADDX1_RVT add_45_U1_2 ( .A(operand_a[2]), .B(shifted_mas[2]), .CI(
    add_45_carry[2]), .CO(add_45_carry[3]), .S(mas_add_result[2]) );
INVX1_RVT U44 ( .A(n28), .Y(n55) );
INVX1_RVT U45 ( .A(n27), .Y(n56) );
INVX1_RVT U46 ( .A(n50), .Y(n49) );
INVX0_RVT U47 ( .A(n50), .Y(n48) );
AO22X1_RVT U48 ( .A1(input_a[6]), .A2(n40), .A3(n41), .A4(n53), .Y(n36) );
INVX1_RVT U49 ( .A(input_b[5]), .Y(n54) );
XNOR3X1_RVT U50 ( .A1(n16), .A2(n45), .A3(n30), .Y(n28) );
XOR2X1_RVT U51 ( .A1(operand_a[4]), .A2(n39), .Y(N30) );
XOR2X1_RVT U52 ( .A1(operand_a[5]), .A2(n38), .Y(N31) );
XNOR2X1_RVT U53 ( .A1(operand_a[6]), .A2(n37), .Y(N32) );
XOR2X1_RVT U54 ( .A1(operand_a[3]), .A2(mas_add_result[4]), .Y(N29) );
NOR2X0_RVT U55 ( .A1(n46), .A2(n47), .Y(n25) );
XNOR3X1_RVT U56 ( .A1(operand_a[6]), .A2(operand_b[6]), .A3(n34), .Y(n46) );
XNOR3X1_RVT U57 ( .A1(operand_a[5]), .A2(operand_b[5]), .A3(n33), .Y(n47) );
INVX1_RVT U58 ( .A(input_a[3]), .Y(n52) );
INVX1_RVT U59 ( .A(input_a[4]), .Y(n51) );
INVX1_RVT U60 ( .A(input_b[6]), .Y(n53) );
INVX1_RVT U61 ( .A(RESETN), .Y(n50) );
OR2X1_RVT U62 ( .A1(shifted_mas[3]), .A2(add_45_carry[3]), .Y(
    mas_add_result[4]) );
XNOR2X1_RVT U63 ( .A1(add_45_carry[3]), .A2(shifted_mas[3]), .Y(
    mas_add_result[3]) );
AND2X1_RVT U64 ( .A1(shifted_mas[0]), .A2(operand_a[0]), .Y(add_45_carry[1])
    );
XOR2X1_RVT U65 ( .A1(operand_a[0]), .A2(shifted_mas[0]), .Y(
    mas_add_result[0]) );
endmodule
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

```

2. FP_8 PART 2:

RTL Code:

```

///////// copy your FP8_ADDER code from part2_p1 then do the retiming based on requirement //////////
module FP8_ADDER(
    CLK,
    RESETN,
    input_a,
    input_b,
    output_c
);

input    CLK;
input    RESETN;
input [7:0] input_a;

```

```

input [7:0] input_b;
output [7:0] output_c;

////////////////////////////////////
// Stage 1 : swap and register operands
////////////////////////////////////
reg [7:0] operand_a;
reg [7:0] operand_b;

always @(posedge CLK or negedge RESETN) begin
    if (!RESETN) begin
        operand_a <= 8'b0;
        operand_b <= 8'b0;
    end
    else begin
        if (input_a[6:3] >= input_b[6:3]) begin
            operand_a <= input_a;
            operand_b <= input_b;
        end
        else begin
            operand_a <= input_b;
            operand_b <= input_a;
        end
    end
end

////////////////////////////////////
// Stage 2 combinational : exponent difference, align, add
////////////////////////////////////
wire [3:0] exp_diff;
assign exp_diff = operand_a[6:3] - operand_b[6:3];

reg [3:0] shifted_mas;
always @(*) begin
    case (exp_diff)
        4'd0: shifted_mas = {1'b1, operand_b[2:0]};
        4'd1: shifted_mas = {1'b0, 1'b1, operand_b[2:1]};
        4'd2: shifted_mas = {2'b00, 1'b1, operand_b[2]};
        4'd3: shifted_mas = {3'b000, 1'b1};
        default: shifted_mas = 4'b0000;
    endcase
end

wire [4:0] mant_sum;
assign mant_sum = {1'b0, 1'b1, operand_a[2:0]} + {1'b0, shifted_mas};

////////////////////////////////////
// Stage 2 register
////////////////////////////////////
reg    s2_sign;
reg [3:0] s2_exp;
reg [4:0] s2_mant_sum;

always @(posedge CLK or negedge RESETN) begin
    if (!RESETN) begin
        s2_sign <= 1'b0;

```

```

        s2_exp    <= 4'b0;
        s2_mant_sum <= 5'b0;
    end
    else begin
        s2_sign    <= operand_a[7];
        s2_exp    <= operand_a[6:3];
        s2_mant_sum <= mant_sum;
    end
end

////////////////////////////////////
// Stage 3 : pack + final register
////////////////////////////////////
wire [7:0] packed_no_carry;
wire [7:0] packed_with_carry;
wire [7:0] packed_result;

assign packed_no_carry = {s2_sign, s2_exp,    s2_mant_sum[2:0]};
assign packed_with_carry = {s2_sign, s2_exp + 1'b1, s2_mant_sum[3:1]};
assign packed_result    = s2_mant_sum[4] ? packed_with_carry
                                     : packed_no_carry;

reg [7:0] sync_result;

always @(posedge CLK or negedge RESETN) begin
    if (!RESETN)
        sync_result <= 8'b0;
    else
        sync_result <= packed_result;
end

assign output_c = sync_result;

```

endmodule

XX

Timing Report at 1.5ns:

Report : timing

-path full

-delay max

-max_paths 5

Design : FP8_ADDER

Version: P-2019.03-SP4

Date : Mon Apr 6 01:06:37 2026

Operating Conditions: tt1p05v125c Library: saed32rvt_tt1p05v125c

Wire Load Model Mode: enclosed

Startpoint: operand_a_reg_3_

(rising edge-triggered flip-flop clocked by CLK)

Endpoint: s2_mant_sum_reg_3_

(rising edge-triggered flip-flop clocked by CLK)

Path Group: CLK

Path Type: max

Des/Clust/Port	Wire Load Model	Library
----------------	-----------------	---------

FP8_ADDER	8000	saed32rvttt1p05v125c
-----------	------	----------------------

Point	Incr	Path

clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
operand_a_reg_3 /CLK (SDFFARX1_RVT)	0.00	0.00 r
operand_a_reg_3 /Q (SDFFARX1_RVT)	0.10	0.10 r
U32/Y (NOR2X0_RVT)	0.07	0.17 f
U30/Y (AND2X1_RVT)	0.05	0.22 f
U29/Y (OAI22X1_RVT)	0.06	0.28 r
U28/Y (OR2X1_RVT)	0.05	0.33 r
U50/Y (XNOR3X1_RVT)	0.07	0.40 f
U27/Y (NOR2X0_RVT)	0.07	0.47 r
U26/Y (AND3X1_RVT)	0.06	0.52 r
U23/Y (AO22X1_RVT)	0.07	0.59 r
U61/Y (AND2X1_RVT)	0.05	0.64 r
add_58_U1_1/CO (FADDX1_RVT)	0.09	0.72 r
add_58_U1_2/CO (FADDX1_RVT)	0.09	0.81 r
U60/Y (XNOR2X1_RVT)	0.09	0.91 r
s2_mant_sum_reg_3 /D (DFFARX1_RVT)	0.01	0.92 r
data arrival time	0.92	
clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
s2_mant_sum_reg_3 /CLK (DFFARX1_RVT)	0.00	1.40 r
library setup time	-0.04	1.36
data required time	1.36	

data required time	1.36	
data arrival time	-0.92	

slack (MET)	0.44	

Startpoint: operand_a_reg_3_
(rising edge-triggered flip-flop clocked by CLK)

Endpoint: s2_mant_sum_reg_4_
(rising edge-triggered flip-flop clocked by CLK)

Path Group: CLK

Path Type: max

Des/Clust/Port	Wire Load Model	Library
----------------	-----------------	---------

FP8_ADDER	8000	saed32rvttt1p05v125c
-----------	------	----------------------

Point	Incr	Path

clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
operand_a_reg_3 /CLK (SDFFARX1_RVT)	0.00	0.00 r
operand_a_reg_3 /Q (SDFFARX1_RVT)	0.10	0.10 r
U32/Y (NOR2X0_RVT)	0.07	0.17 f

U30/Y (AND2X1_RVT)	0.05	0.22 f
U29/Y (OAI22X1_RVT)	0.06	0.28 r
U28/Y (OR2X1_RVT)	0.05	0.33 r
U50/Y (XNOR3X1_RVT)	0.07	0.40 f
U27/Y (NOR2X0_RVT)	0.07	0.47 r
U26/Y (AND3X1_RVT)	0.06	0.52 r
U23/Y (AO22X1_RVT)	0.07	0.59 r
U61/Y (AND2X1_RVT)	0.05	0.64 r
add_58_U1_1/CO (FADDX1_RVT)	0.09	0.72 r
add_58_U1_2/CO (FADDX1_RVT)	0.09	0.81 r
U59/Y (OR2X1_RVT)	0.05	0.86 r
s2_mant_sum_reg_4_/D (DFFARX1_RVT)	0.01	0.88 r
data arrival time	0.88	

clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
s2_mant_sum_reg_4_/CLK (DFFARX1_RVT)	0.00	1.40 r
library setup time	-0.03	1.37
data required time	1.37	

data required time	1.37
data arrival time	-0.88

slack (MET)	0.49
-------------	------

Startpoint: operand_a_reg_3_
(rising edge-triggered flip-flop clocked by CLK)
Endpoint: s2_mant_sum_reg_2_
(rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max

Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvtt1p05v125c

Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
operand_a_reg_3_/CLK (SDFFARX1_RVT)	0.00	0.00 r
operand_a_reg_3_/Q (SDFFARX1_RVT)	0.10	0.10 r
U32/Y (NOR2X0_RVT)	0.07	0.17 f
U30/Y (AND2X1_RVT)	0.05	0.22 f
U29/Y (OAI22X1_RVT)	0.06	0.28 r
U28/Y (OR2X1_RVT)	0.05	0.33 r
U50/Y (XNOR3X1_RVT)	0.07	0.40 r
U27/Y (NOR2X0_RVT)	0.07	0.47 f
U26/Y (AND3X1_RVT)	0.06	0.53 f
U23/Y (AO22X1_RVT)	0.06	0.59 f
U61/Y (AND2X1_RVT)	0.05	0.64 f
add_58_U1_1/CO (FADDX1_RVT)	0.07	0.72 f
add_58_U1_2/S (FADDX1_RVT)	0.11	0.83 r
s2_mant_sum_reg_2_/D (DFFARX1_RVT)	0.01	0.84 r

data arrival time	0.84		
clock CLK (rise edge)	1.50	1.50	
clock network delay (ideal)	0.00	1.50	
clock uncertainty	-0.10	1.40	
s2_mant_sum_reg_2_/CLK (DFFARX1_RVT)	0.00	1.40	r
library setup time	-0.04	1.36	
data required time	1.36		

data required time	1.36		
data arrival time	-0.84		

slack (MET)	0.52		

Startpoint: operand_a_reg_3_
(rising edge-triggered flip-flop clocked by CLK)
Endpoint: s2_mant_sum_reg_1_
(rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max

Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvt_tt1p05v125c

Point	Incr	Path	

clock CLK (rise edge)	0.00	0.00	
clock network delay (ideal)	0.00	0.00	
operand_a_reg_3_/CLK (SDFFARX1_RVT)	0.00	0.00	r
operand_a_reg_3_/Q (SDFFARX1_RVT)	0.10	0.10	r
U32/Y (NOR2X0_RVT)	0.07	0.17	f
U30/Y (AND2X1_RVT)	0.05	0.22	f
U29/Y (OAI22X1_RVT)	0.06	0.28	r
U28/Y (OR2X1_RVT)	0.05	0.33	r
U50/Y (XNOR3X1_RVT)	0.07	0.40	r
U27/Y (NOR2X0_RVT)	0.07	0.47	f
U26/Y (AND3X1_RVT)	0.06	0.53	f
U23/Y (AO22X1_RVT)	0.06	0.59	f
U61/Y (AND2X1_RVT)	0.05	0.64	f
add_58_U1_1/S (FADDX1_RVT)	0.11	0.75	r
s2_mant_sum_reg_1_/D (DFFARX1_RVT)	0.01	0.76	r
data arrival time	0.76		
clock CLK (rise edge)	1.50	1.50	
clock network delay (ideal)	0.00	1.50	
clock uncertainty	-0.10	1.40	
s2_mant_sum_reg_1_/CLK (DFFARX1_RVT)	0.00	1.40	r
library setup time	-0.04	1.36	
data required time	1.36		

data required time	1.36		
data arrival time	-0.76		

slack (MET)	0.60		

Startpoint: operand_a_reg_3_
 (rising edge-triggered flip-flop clocked by CLK)
 Endpoint: s2_mant_sum_reg_0_
 (rising edge-triggered flip-flop clocked by CLK)
 Path Group: CLK
 Path Type: max

Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvt_tt1p05v125c

Point	Incr	Path

clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
operand_a_reg_3_/CLK (SDFFARX1_RVT)	0.00	0.00 r
operand_a_reg_3_/Q (SDFFARX1_RVT)	0.10	0.10 r
U32/Y (NOR2X0_RVT)	0.07	0.17 f
U30/Y (AND2X1_RVT)	0.05	0.22 f
U29/Y (OAI22X1_RVT)	0.06	0.28 r
U28/Y (OR2X1_RVT)	0.05	0.33 r
U50/Y (XNOR3X1_RVT)	0.07	0.40 r
U27/Y (NOR2X0_RVT)	0.07	0.47 f
U26/Y (AND3X1_RVT)	0.06	0.53 f
U23/Y (AO22X1_RVT)	0.06	0.59 f
U62/Y (XOR2X1_RVT)	0.09	0.69 r
s2_mant_sum_reg_0_/D (DFFARX1_RVT)	0.01	0.70 r
data arrival time	0.70	
clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
s2_mant_sum_reg_0_/CLK (DFFARX1_RVT)	0.00	1.40 r
library setup time	-0.04	1.36
data required time	1.36	

data required time	1.36	
data arrival time	-0.70	

slack (MET)	0.66	

1

XX

Area Report at 1.5ns:

Report : area

Design : FP8_ADDER

Version: P-2019.03-SP4

Date : Mon Apr 6 01:06:37 2026

Library(s) Used:

saed32rvt_tt1p05v125c (File: /app11/saed_32.28/SAED32_EDK/lib/stdcell_rvt/db_ccs/saed32rvt_tt1p05v125c.db)

Number of ports:	26
Number of nets:	102
Number of cells:	80
Number of combinational cells:	45
Number of sequential cells:	33
Number of macros/black boxes:	0
Number of buf/inv:	9
Number of references:	18

Combinational area:	123.513984
Buf/Inv area:	11.436480
Noncombinational area:	271.425798
Macro/Black Box area:	0.000000
Net Interconnect area:	61.508674

Total cell area:	394.939782
Total area:	456.448456
1	

XX
XX

3. FP_8 PART 3:

RTL Code:

```

///////// copy your FP8_ADDER code from part2_p1 then do the retiming based on requirement ///////////
module FP8_ADDER(
  CLK,
  RESETN,
  input_a,
  input_b,
  output_c
);

input CLK;
input RESETN;
input [7:0] input_a;
input [7:0] input_b;
output [7:0] output_c;

////////////////////////////////////
//// Stage 1 : register raw inputs
////////////////////////////////////
reg [7:0] in_a_r;
reg [7:0] in_b_r;

always @(posedge CLK or negedge RESETN) begin
  if (!RESETN) begin
    in_a_r <= 8'b0;
    in_b_r <= 8'b0;
  end
  else begin
    in_a_r <= input_a;
    in_b_r <= input_b;
  end
end

////////////////////////////////////
//// Stage 2 : compare + select fields

```

```

////////////////////////////////////
wire sel_ab_w;
wire [3:0] diff_abs_w;
wire      sign_big_w;
wire [3:0] exp_big_w;
wire [3:0] mant_big_w;
wire [3:0] mant_small_w;

assign sel_ab_w  = (in_a_r[6:3] >= in_b_r[6:3]);
assign diff_abs_w = sel_ab_w ? (in_a_r[6:3] - in_b_r[6:3])
                             : (in_b_r[6:3] - in_a_r[6:3]);

assign sign_big_w = sel_ab_w ? in_a_r[7]      : in_b_r[7];
assign exp_big_w  = sel_ab_w ? in_a_r[6:3]    : in_b_r[6:3];
assign mant_big_w = sel_ab_w ? {1'b1, in_a_r[2:0]} : {1'b1, in_b_r[2:0]};
assign mant_small_w = sel_ab_w ? {1'b1, in_b_r[2:0]} : {1'b1, in_a_r[2:0]};

reg      s2_sign;
reg [3:0] s2_exp_big;
reg [3:0] s2_mant_big;
reg [3:0] s2_mant_small;
reg [3:0] s2_diff_abs;

always @(posedge CLK or negedge RESETN) begin
    if (!RESETN) begin
        s2_sign    <= 1'b0;
        s2_exp_big <= 4'b0;
        s2_mant_big <= 4'b0;
        s2_mant_small <= 4'b0;
        s2_diff_abs <= 4'b0;
    end
    else begin
        s2_sign    <= sign_big_w;
        s2_exp_big <= exp_big_w;
        s2_mant_big <= mant_big_w;
        s2_mant_small <= mant_small_w;
        s2_diff_abs <= diff_abs_w;
    end
end

////////////////////////////////////
/// Stage 3 : align + add
////////////////////////////////////
reg [3:0] shifted_small_w;
always @(*) begin
    case (s2_diff_abs)
        4'd0: shifted_small_w = s2_mant_small;
        4'd1: shifted_small_w = {1'b0, s2_mant_small[3:1]};
        4'd2: shifted_small_w = {2'b00, s2_mant_small[3:2]};
        4'd3: shifted_small_w = {3'b000, s2_mant_small[3]};
        default: shifted_small_w = 4'b0000;
    endcase
end

wire [4:0] mant_sum_w;
assign mant_sum_w = {1'b0, s2_mant_big} + {1'b0, shifted_small_w};

```

```

reg    s3_sign;
reg [3:0] s3_exp_big;
reg [4:0] s3_mant_sum;

always @(posedge CLK or negedge RESETN) begin
    if (!RESETN) begin
        s3_sign    <= 1'b0;
        s3_exp_big <= 4'b0;
        s3_mant_sum <= 5'b0;
    end
    else begin
        s3_sign    <= s2_sign;
        s3_exp_big <= s2_exp_big;
        s3_mant_sum <= mant_sum_w;
    end
end

//////////
//// Stage 4 : normalize + pack + output reg
//////////
wire [7:0] packed_no_carry_w;
wire [7:0] packed_with_carry_w;
wire [7:0] packed_result_w;

assign packed_no_carry_w = {s3_sign, s3_exp_big,    s3_mant_sum[2:0]};
assign packed_with_carry_w = {s3_sign, s3_exp_big + 1'b1, s3_mant_sum[3:1]};
assign packed_result_w    = s3_mant_sum[4] ? packed_with_carry_w
                                         : packed_no_carry_w;

reg [7:0] sync_result;

always @(posedge CLK or negedge RESETN) begin
    if (!RESETN)
        sync_result <= 8'b0;
    else
        sync_result <= packed_result_w;
end

assign output_c = sync_result;

```

endmodule

XX

Timing Report at 1.5 ns:

Report : timing

-path full

-delay max

-max_paths 5

Design : FP8_ADDER

Version: P-2019.03-SP4

Date : Mon Apr 6 01:13:02 2026

Operating Conditions: tt1p05v125c Library: saed32rvt_tt1p05v125c

Wire Load Model Mode: enclosed

Startpoint: s2_diff_abs_reg_3_
 (rising edge-triggered flip-flop clocked by CLK)
 Endpoint: s3_mant_sum_reg_3_
 (rising edge-triggered flip-flop clocked by CLK)
 Path Group: CLK
 Path Type: max

Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvt_tt1p05v125c

Point	Incr	Path		
clock CLK (rise edge)	0.00	0.00		
clock network delay (ideal)	0.00	0.00		
s2_diff_abs_reg_3_/CLK (DFFARX1_RVT)		0.00	0.00	r
s2_diff_abs_reg_3_/Q (DFFARX1_RVT)		0.09	0.09	r
U33/Y (NOR2X0_RVT)	0.06	0.16	f	
U32/Y (AND2X1_RVT)	0.05	0.21	f	
U28/Y (NAND2X0_RVT)	0.04	0.25	r	
U65/Y (INVX1_RVT)	0.03	0.29	f	
U27/Y (NAND2X0_RVT)	0.03	0.32	r	
U26/Y (NAND3X0_RVT)	0.05	0.37	f	
U82/Y (AND2X1_RVT)	0.06	0.43	f	
add_91_U1_1/CO (FADDX1_RVT)		0.08	0.50	f
add_91_U1_2/CO (FADDX1_RVT)		0.08	0.58	f
add_91_U1_3/S (FADDX1_RVT)		0.11	0.69	r
s3_mant_sum_reg_3_/D (DFFARX1_RVT)		0.01	0.70	r
data arrival time	0.70			
clock CLK (rise edge)	1.50	1.50		
clock network delay (ideal)	0.00	1.50		
clock uncertainty	-0.10	1.40		
s3_mant_sum_reg_3_/CLK (DFFARX1_RVT)		0.00	1.40	r
library setup time	-0.04	1.36		
data required time	1.36			
data required time	1.36			
data arrival time	-0.70			
slack (MET)	0.66			

Startpoint: s2_diff_abs_reg_3_
 (rising edge-triggered flip-flop clocked by CLK)
 Endpoint: s3_mant_sum_reg_4_
 (rising edge-triggered flip-flop clocked by CLK)
 Path Group: CLK
 Path Type: max

Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvt_tt1p05v125c

Point	Incr	Path
-------	------	------

clock CLK (rise edge)	0.00	0.00	
clock network delay (ideal)	0.00	0.00	
s2_diff_abs_reg_3_/CLK (DFFARX1_RVT)		0.00	0.00 r
s2_diff_abs_reg_3_/Q (DFFARX1_RVT)		0.09	0.09 f
U33/Y (NOR2X0_RVT)	0.06	0.15	r
U32/Y (AND2X1_RVT)	0.05	0.20	r
U28/Y (NAND2X0_RVT)	0.04	0.25	f
U65/Y (INVX1_RVT)	0.04	0.28	r
U27/Y (NAND2X0_RVT)	0.03	0.31	f
U26/Y (NAND3X0_RVT)	0.04	0.35	r
U82/Y (AND2X1_RVT)	0.05	0.40	r
add_91_U1_1/CO (FADDX1_RVT)		0.09	0.48 r
add_91_U1_2/CO (FADDX1_RVT)		0.09	0.57 r
add_91_U1_3/CO (FADDX1_RVT)		0.08	0.65 r
s3_mant_sum_reg_4_/D (DFFARX1_RVT)		0.01	0.67 r
data arrival time	0.67		
clock CLK (rise edge)	1.50	1.50	
clock network delay (ideal)	0.00	1.50	
clock uncertainty	-0.10	1.40	
s3_mant_sum_reg_4_/CLK (DFFARX1_RVT)		0.00	1.40 r
library setup time	-0.04	1.36	
data required time	1.36		

data required time	1.36		
data arrival time	-0.67		

slack (MET)	0.70		

Startpoint: s2_diff_abs_reg_3_
(rising edge-triggered flip-flop clocked by CLK)

Endpoint: s3_mant_sum_reg_2_
(rising edge-triggered flip-flop clocked by CLK)

Path Group: CLK

Path Type: max

Des/Clust/Port Wire Load Model Library

FP8_ADDER 8000 saed32rvt_tt1p05v125c

Point	Incr	Path

clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
s2_diff_abs_reg_3_/CLK (DFFARX1_RVT)		0.00 0.00 r
s2_diff_abs_reg_3_/Q (DFFARX1_RVT)		0.09 0.09 r
U33/Y (NOR2X0_RVT)	0.06	0.16 f
U32/Y (AND2X1_RVT)	0.05	0.21 f
U28/Y (NAND2X0_RVT)	0.04	0.25 r
U65/Y (INVX1_RVT)	0.03	0.29 f
U27/Y (NAND2X0_RVT)	0.03	0.32 r
U26/Y (NAND3X0_RVT)	0.05	0.37 f
U82/Y (AND2X1_RVT)	0.06	0.43 f
add_91_U1_1/CO (FADDX1_RVT)		0.08 0.50 f

add_91_U1_2/S (FADDX1_RVT)	0.11	0.62 r
s3_mant_sum_reg_2_/D (DFFARX1_RVT)	0.01	0.63 r
data arrival time	0.63	
clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
s3_mant_sum_reg_2_/CLK (DFFARX1_RVT)	0.00	1.40 r
library setup time	-0.04	1.36
data required time	1.36	

data required time	1.36	
data arrival time	-0.63	

slack (MET)	0.74	

Startpoint: in_b_r_reg_4_
(rising edge-triggered flip-flop clocked by CLK)
Endpoint: s2_diff_abs_reg_3_
(rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max

Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvt_tt1p05v125c

Point	Incr	Path

clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
in_b_r_reg_4_/CLK (DFFARX1_RVT)	0.00	0.00 r
in_b_r_reg_4_/Q (DFFARX1_RVT)	0.10	0.10 r
U56/Y (OR2X1_RVT)	0.06	0.16 r
U51/Y (NAND3X0_RVT)	0.04	0.20 f
U50/Y (NAND3X0_RVT)	0.04	0.24 r
U49/Y (NAND2X0_RVT)	0.04	0.27 f
U48/Y (OR2X1_RVT)	0.05	0.32 f
U70/Y (AO22X1_RVT)	0.09	0.41 f
U64/Y (INVX1_RVT)	0.06	0.47 r
U38/Y (NAND3X0_RVT)	0.04	0.51 f
U37/Y (NAND2X0_RVT)	0.04	0.55 r
s2_diff_abs_reg_3_/D (DFFARX1_RVT)	0.01	0.56 r
data arrival time	0.56	
clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
s2_diff_abs_reg_3_/CLK (DFFARX1_RVT)	0.00	1.40 r
library setup time	-0.04	1.36
data required time	1.36	

data required time	1.36	
data arrival time	-0.56	

slack (MET) 0.80

Startpoint: s2_diff_abs_reg_3_
(rising edge-triggered flip-flop clocked by CLK)
Endpoint: s3_mant_sum_reg_1_
(rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max

Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvt_tt1p05v125c

Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
s2_diff_abs_reg_3_/CLK (DFFARX1_RVT)		0.00 0.00 r
s2_diff_abs_reg_3_/Q (DFFARX1_RVT)		0.09 0.09 r
U33/Y (NOR2X0_RVT)	0.06	0.16 f
U32/Y (AND2X1_RVT)	0.05	0.21 f
U28/Y (NAND2X0_RVT)	0.04	0.25 r
U65/Y (INVX1_RVT)	0.03	0.29 f
U27/Y (NAND2X0_RVT)	0.03	0.32 r
U26/Y (NAND3X0_RVT)	0.05	0.37 f
U82/Y (AND2X1_RVT)	0.06	0.43 f
add_91_U1_1/S (FADDX1_RVT)		0.11 0.54 r
s3_mant_sum_reg_1_/D (DFFARX1_RVT)		0.01 0.55 r
data arrival time	0.55	
clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
s3_mant_sum_reg_1_/CLK (DFFARX1_RVT)		0.00 1.40 r
library setup time	-0.04	1.36
data required time	1.36	
data required time	1.36	
data arrival time	-0.55	
slack (MET)	0.81	

1

XX

Area Report at 1.5 ns:

Report : area

Design : FP8_ADDER

Version: P-2019.03-SP4

Date : Mon Apr 6 01:13:02 2026

Library(s) Used:

saed32rvt_tt1p05v125c (File: /app11/saed_32.28/SAED32_EDK/lib/stdcell_rvt/db_ccs/saed32rvt_tt1p05v125c.db)

Number of ports:	26
Number of nets:	146
Number of cells:	117
Number of combinational cells:	64
Number of sequential cells:	51
Number of macros/black boxes:	0
Number of buf/inv:	11
Number of references:	18

Combinational area:	150.453248
Buf/Inv area:	16.265216
Noncombinational area:	395.448072
Macro/Black Box area:	0.000000
Net Interconnect area:	94.283954

Total cell area:	545.901320
Total area:	640.185274

1

XX
XX

4. FP_8 PART 4:

RTL Code:

```

//////// copy your ADDER code from p1 and fully pipeline it according to the requirement //////////
module FP8_ADDER(
  CLK,
  RESETN,
  input_a,
  input_b,
  output_c
);

input CLK;
input RESETN;
input [7:0] input_a;
input [7:0] input_b;
output [7:0] output_c;

//////////
//// Stage 1 : register raw inputs
//////////
reg [7:0] in_a_r;
reg [7:0] in_b_r;

always @(posedge CLK or negedge RESETN) begin
  if (!RESETN) begin
    in_a_r <= 8'b0;
    in_b_r <= 8'b0;
  end
  else begin
    in_a_r <= input_a;
    in_b_r <= input_b;
  end
end

//////////

```

```

//// Stage 2 : compare + select narrow fields only
////////////////////////////////////////
wire sel_ab_w;
wire [3:0] diff_abs_w;
wire sign_big_w;
wire [3:0] exp_big_w;
wire [3:0] mant_big_w;
wire [3:0] mant_small_w;

assign sel_ab_w = (in_a_r[6:3] >= in_b_r[6:3]);
assign diff_abs_w = sel_ab_w ? (in_a_r[6:3] - in_b_r[6:3])
                             : (in_b_r[6:3] - in_a_r[6:3]);

assign sign_big_w = sel_ab_w ? in_a_r[7] : in_b_r[7];
assign exp_big_w = sel_ab_w ? in_a_r[6:3] : in_b_r[6:3];
assign mant_big_w = sel_ab_w ? {1'b1, in_a_r[2:0]} : {1'b1, in_b_r[2:0]};
assign mant_small_w = sel_ab_w ? {1'b1, in_b_r[2:0]} : {1'b1, in_a_r[2:0]};

reg s2_sign;
reg [3:0] s2_exp_big;
reg [3:0] s2_mant_big;
reg [3:0] s2_mant_small;
reg [3:0] s2_diff_abs;

always @(posedge CLK or negedge RESETN) begin
    if (!RESETN) begin
        s2_sign <= 1'b0;
        s2_exp_big <= 4'b0;
        s2_mant_big <= 4'b0;
        s2_mant_small <= 4'b0;
        s2_diff_abs <= 4'b0;
    end
    else begin
        s2_sign <= sign_big_w;
        s2_exp_big <= exp_big_w;
        s2_mant_big <= mant_big_w;
        s2_mant_small <= mant_small_w;
        s2_diff_abs <= diff_abs_w;
    end
end

////////////////////////////////////////
//// Stage 3 : alignment only
////////////////////////////////////////
reg [3:0] shifted_small_w;
always @(*) begin
    case (s2_diff_abs)
        4'd0: shifted_small_w = s2_mant_small;
        4'd1: shifted_small_w = {1'b0, s2_mant_small[3:1]};
        4'd2: shifted_small_w = {2'b00, s2_mant_small[3:2]};
        4'd3: shifted_small_w = {3'b000, s2_mant_small[3]};
        default: shifted_small_w = 4'b0000;
    endcase
end

reg s3_sign;

```

```

reg [3:0] s3_exp_big;
reg [3:0] s3_mant_big;
reg [3:0] s3_shifted_small;

always @(posedge CLK or negedge RESETN) begin
    if (!RESETN) begin
        s3_sign      <= 1'b0;
        s3_exp_big   <= 4'b0;
        s3_mant_big  <= 4'b0;
        s3_shifted_small <= 4'b0;
    end
    else begin
        s3_sign      <= s2_sign;
        s3_exp_big   <= s2_exp_big;
        s3_mant_big  <= s2_mant_big;
        s3_shifted_small <= shifted_small_w;
    end
end

////////////////////////////////////
/// Stage 4 : mantissa add only
////////////////////////////////////
wire [4:0] mant_sum_w;
assign mant_sum_w = {1'b0, s3_mant_big} + {1'b0, s3_shifted_small};

reg    s4_sign;
reg [3:0] s4_exp_big;
reg [4:0] s4_mant_sum;

always @(posedge CLK or negedge RESETN) begin
    if (!RESETN) begin
        s4_sign  <= 1'b0;
        s4_exp_big <= 4'b0;
        s4_mant_sum <= 5'b0;
    end
    else begin
        s4_sign  <= s3_sign;
        s4_exp_big <= s3_exp_big;
        s4_mant_sum <= mant_sum_w;
    end
end

////////////////////////////////////
/// Stage 5 : normalize + pack + output reg
////////////////////////////////////
wire [7:0] packed_no_carry_w;
wire [7:0] packed_with_carry_w;
wire [7:0] packed_result_w;

assign packed_no_carry_w  = {s4_sign, s4_exp_big,    s4_mant_sum[2:0]};
assign packed_with_carry_w = {s4_sign, s4_exp_big + 1'b1, s4_mant_sum[3:1]};
assign packed_result_w    = s4_mant_sum[4] ? packed_with_carry_w
                                           : packed_no_carry_w;

reg [7:0] sync_result;

```


clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
s2_diff_abs_reg_3_/CLK (DFFARX1_RVT)	0.00	1.40 r
library setup time	-0.04	1.36
data required time	1.36	

data required time	1.36
data arrival time	-0.56

slack (MET)	0.80
-------------	------

Startpoint: in_b_r_reg_4_
(rising edge-triggered flip-flop clocked by CLK)

Endpoint: s2_diff_abs_reg_2_
(rising edge-triggered flip-flop clocked by CLK)

Path Group: CLK

Path Type: max

Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvtt1p05v125c

Point	Incr	Path
-------	------	------

clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
in_b_r_reg_4_/CLK (DFFARX1_RVT)		0.00 0.00 r
in_b_r_reg_4_/Q (DFFARX1_RVT)		0.10 0.10 f
U57/Y (OR2X1_RVT)	0.06	0.16 f
U52/Y (NAND3X0_RVT)	0.04	0.19 r
U51/Y (NAND3X0_RVT)	0.04	0.23 f
U50/Y (NAND2X0_RVT)	0.04	0.28 r
U49/Y (OR2X1_RVT)	0.05	0.33 r
U72/Y (AO22X1_RVT)	0.09	0.42 r
s2_diff_abs_reg_2_/SE (SDFFARX1_RVT)		0.05 0.47 r
data arrival time	0.47	

clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
s2_diff_abs_reg_2_/CLK (SDFFARX1_RVT)		0.00 1.40 r
library setup time	-0.08	1.32
data required time	1.32	

data required time	1.32
data arrival time	-0.47

slack (MET)	0.85
-------------	------

Startpoint: in_b_r_reg_4_
(rising edge-triggered flip-flop clocked by CLK)

Endpoint: s2_diff_abs_reg_1_
(rising edge-triggered flip-flop clocked by CLK)

Path Group: CLK

Path Type: max

Des/Clust/Port Wire Load Model Library

FP8_ADDER 8000 saed32rvt_tt1p05v125c

Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
in_b_r_reg_4_/CLK (DFFARX1_RVT)		0.00 0.00 r
in_b_r_reg_4_/Q (DFFARX1_RVT)		0.10 0.10 f
U57/Y (OR2X1_RVT)	0.06	0.16 f
U52/Y (NAND3X0_RVT)	0.04	0.19 r
U51/Y (NAND3X0_RVT)	0.04	0.23 f
U50/Y (NAND2X0_RVT)	0.04	0.28 r
U49/Y (OR2X1_RVT)	0.05	0.33 r
U72/Y (AO22X1_RVT)	0.09	0.42 r
s2_diff_abs_reg_1_/SE (SDDFFARX1_RVT)		0.05 0.47 r
data arrival time	0.47	
clock CLK (rise edge)	1.50	1.50
clock network delay (ideal)	0.00	1.50
clock uncertainty	-0.10	1.40
s2_diff_abs_reg_1_/CLK (SDFFARX1_RVT)		0.00 1.40 r
library setup time	-0.08	1.32
data required time	1.32	
data required time	1.32	
data arrival time	-0.47	
slack (MET)	0.85	

Startpoint: in_b_r_reg_4_

(rising edge-triggered flip-flop clocked by CLK)

Endpoint: s2_exp_big_reg_1_

(rising edge-triggered flip-flop clocked by CLK)

Path Group: CLK

Path Type: max

Des/Clust/Port Wire Load Model Library

FP8_ADDER 8000 saed32rvt_tt1p05v125c

Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
in_b_r_reg_4_/CLK (DFFARX1_RVT)		0.00 0.00 r
in_b_r_reg_4_/Q (DFFARX1_RVT)		0.10 0.10 f
U57/Y (OR2X1_RVT)	0.06	0.16 f
U52/Y (NAND3X0_RVT)	0.04	0.19 r
U51/Y (NAND3X0_RVT)	0.04	0.23 f
U50/Y (NAND2X0_RVT)	0.04	0.28 r
U49/Y (OR2X1_RVT)	0.05	0.33 r

U72/Y (AO22X1_RVT)	0.09	0.42 r	
s2_exp_big_reg_1_/SE (SDFFARX1_RVT)	0.05	0.47 r	
data arrival time	0.47		

clock CLK (rise edge)	1.50	1.50	
clock network delay (ideal)	0.00	1.50	
clock uncertainty	-0.10	1.40	
s2_exp_big_reg_1_/CLK (SDFFARX1_RVT)	0.00	1.40 r	
library setup time	-0.08	1.32	
data required time	1.32		

data required time	1.32		
data arrival time	-0.47		

slack (MET)	0.85		

Startpoint: in_b_r_reg_4_
(rising edge-triggered flip-flop clocked by CLK)
Endpoint: s2_exp_big_reg_0_
(rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max

Des/Clust/Port	Wire Load Model	Library
FP8_ADDER	8000	saed32rvtt1p05v125c

Point	Incr	Path	

clock CLK (rise edge)	0.00	0.00	
clock network delay (ideal)	0.00	0.00	
in_b_r_reg_4_/CLK (DFFARX1_RVT)		0.00	0.00 r
in_b_r_reg_4_/Q (DFFARX1_RVT)		0.10	0.10 f
U57/Y (OR2X1_RVT)	0.06	0.16 f	
U52/Y (NAND3X0_RVT)	0.04	0.19 r	
U51/Y (NAND3X0_RVT)	0.04	0.23 f	
U50/Y (NAND2X0_RVT)	0.04	0.28 r	
U49/Y (OR2X1_RVT)	0.05	0.33 r	
U72/Y (AO22X1_RVT)	0.09	0.42 r	
s2_exp_big_reg_0_/SE (SDFFARX1_RVT)	0.05	0.47 r	
data arrival time	0.47		

clock CLK (rise edge)	1.50	1.50	
clock network delay (ideal)	0.00	1.50	
clock uncertainty	-0.10	1.40	
s2_exp_big_reg_0_/CLK (SDFFARX1_RVT)	0.00	1.40 r	
library setup time	-0.08	1.32	
data required time	1.32		

data required time	1.32		
data arrival time	-0.47		

slack (MET)	0.85		

XX

Area Report at 1.5 ns:

Report : area

Design : FP8_ADDER

Version: P-2019.03-SP4

Date : Mon Apr 6 01:29:10 2026

Library(s) Used:

saed32rvt_tt1p05v125c (File: /app11/saed_32.28/SAED32_EDK/lib/stdcell_rvt/db_ccs/saed32rvt_tt1p05v125c.db)

Number of ports:	26
Number of nets:	161
Number of cells:	132
Number of combinational cells:	65
Number of sequential cells:	64
Number of macros/black boxes:	0
Number of buf/inv:	13
Number of references:	18

Combinational area:	152.486400
Buf/Inv area:	20.331521
Noncombinational area:	489.989642
Macro/Black Box area:	0.000000
Net Interconnect area:	110.280410

Total cell area:	642.476042
Total area:	752.756452

1

XX
XX

Appendix B

Test Bench:

```
//// testbench is where your test the design by mimic the signal from PCB ////
//// in part2_p2, you could copy testbench from p1 ////

`timescale 1ns/10ps

module tb_FP8_ADDER();

parameter period = 2;
parameter num_of_case = 256;
integer i;
integer j;
integer num_of_error;

//// do touch below ////
reg [0:0]          clk;
reg [0:0]          cs;
reg [0:0]          we;
reg [7:0]          addr;
reg [127:0]        din;
wire[127:0]        dout;

MockSRAM #(.DEPTH(256), .ADDR_WIDTH(8), .DATA_WIDTH(128)) SRAMBANKA0 ( .clk(clk), .cs(cs),
.we(we), .addr(addr), .din(din), .dout(dout) );
reg [7:0] output_bits_reference [0:255];
always #(period/2.0) clk=~clk;

initial begin
    $readmemb("../ref/SRAM_DATA0.txt", SRAMBANKA0.memory);
    $readmemb("../ref/output_ans.txt", output_bits_reference);
end

////don't touch above ////

wire [7:0] input_0;    //// first fp8 operand
wire [7:0] input_1;    //// second fp8 operand
wire [7:0] c;          //// fp8 output

reg [0:0] RESETN;      //// reset disable

//////// assign input here, dout is 16-bit input read from sram////////
assign input_0 = dout[15:8];
assign input_1 = dout[7:0];

//// instantiate your ADDER here ////
FP8_ADDER U_FP8_ADDER (
    .CLK(clk),
    .RESETN(RESETN),
    .input_a(input_0),
    .input_b(input_1),
    .output_c(c)
);

//// don't touch below ////
```

[illegible]

```

elaborate $design_name

source ./constraint.tcl

set_fix_multiple_port_nets -all -exclude_clock_network -buffer_constants [get_designs *]

link

compile
#optimize_registers -print_critical_loop -minimum_period_only
remove_unconnected_ports -blast_buses [get_cells "*" -hier]

change_name -hierarchy -rules verilog

write -format verilog -hierarchy -output ./output/$design_name.v
write_sdf ./output/$design_name.sdf
write_sdc ./output/$design_name.sdc
write -hierarchy -output ./output/$design_name.ddc

check_design > ./report/check_$design_name.rpt
report_qor > ./report/qor_$design_name.rpt
report_timing -max_path $maxpaths > ./report/timing_$design_name.rpt
report_area > ./report/area_$design_name.rpt

exit
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

```

Constraint.tcl code:

```

# Synthesis Constraints for FP8_ADDER

# If CLK_PER is not passed from dc-syn.tcl / command line, use default 1.5 ns
if {[info exists CLK_PER]} {
    set CLK_PER 1.5
}

# Clock
create_clock -period $CLK_PER [get_ports CLK]
set_clock_uncertainty 0.1 [get_clocks CLK]

# Input transition (exclude clock)
set_input_transition 0.1 [remove_from_collection [all_inputs] [get_ports CLK]]

# Input delay (exclude clock and reset)
set_input_delay 0.2 -clock CLK [remove_from_collection [all_inputs] [get_ports {CLK RESETN}]]

# Output delay
set_output_delay 0.2 -clock CLK [all_outputs]

# Output load capacitance = 5 fF
set_load 0.005 [all_outputs]

# Driving gate = inverter with 4X strength
set_driving_cell -lib_cell INVX4_RVT [remove_from_collection [all_inputs] [get_ports CLK]]

# Max area = 0 (no constraint)
set_max_area 0

```

XX

Clock Sweep.sh Code:

```
#!/bin/bash

start=1.5
step=0.1
min=0.1

DC_BIN="/app11/synopsys/syn/P-2019.03-SP4/bin/dc_shell-xg-t"
RESULTS="sweep_results.csv"
ARCHIVE_DIR="sweep_archive"

mkdir -p output report "$ARCHIVE_DIR"

echo "Clock(ns),Arrival(ns),Slack(ns),Status,Area" > "$RESULTS"

p=$start

while awk "BEGIN {exit !($p >= $min)}"
do
    printf "Running synthesis for clock period = %.1f ns\n" "$p"

    # update CLK_PER in constraint.tcl
    sed -i "s/^[[:space:]]*set CLK_PER .* / set CLK_PER $p/" constraint.tcl

    # clean only the base files that dc-syn.tcl regenerates
    rm -f report/timing_FP8_ADDER.rpt
    rm -f report/area_FP8_ADDER.rpt
    rm -f report/check_FP8_ADDER.rpt
    rm -f report/qor_FP8_ADDER.rpt

    rm -f output/FP8_ADDER.v
    rm -f output/FP8_ADDER.sdf
    rm -f output/FP8_ADDER.sdc
    rm -f output/FP8_ADDER.ddc

    # tag like 1p5, 1p4, 0p9
    tag=$(printf "%.1f" "$p" | sed 's/\./p/g')

    # run synthesis
    "$DC_BIN" -f dc-syn.tcl > run_${tag}.log

    # archive reports and outputs
    cp run_${tag}.log "$ARCHIVE_DIR/run_${tag}.log"
    cp report/timing_FP8_ADDER.rpt "$ARCHIVE_DIR/timing_${tag}.rpt"
    cp report/area_FP8_ADDER.rpt "$ARCHIVE_DIR/area_${tag}.rpt"
    cp report/check_FP8_ADDER.rpt "$ARCHIVE_DIR/check_${tag}.rpt" 2>/dev/null
    cp report/qor_FP8_ADDER.rpt "$ARCHIVE_DIR/qor_${tag}.rpt" 2>/dev/null

    cp output/FP8_ADDER.v "$ARCHIVE_DIR/FP8_ADDER_${tag}.v" 2>/dev/null
    cp output/FP8_ADDER.sdf "$ARCHIVE_DIR/FP8_ADDER_${tag}.sdf" 2>/dev/null
    cp output/FP8_ADDER.sdc "$ARCHIVE_DIR/FP8_ADDER_${tag}.sdc" 2>/dev/null
    cp output/FP8_ADDER.ddc "$ARCHIVE_DIR/FP8_ADDER_${tag}.ddc" 2>/dev/null

    # extract values
    arrival=$(grep -i "data arrival time" "$ARCHIVE_DIR/timing_${tag}.rpt" | head -1 | awk '{print $4}')
```

```

slack=$(grep -i "slack" "$ARCHIVE_DIR/timing_${tag}.rpt" | head -1 | awk '{print $3}')
area=$(grep -i "Total area:" "$ARCHIVE_DIR/area_${tag}.rpt" | head -1 | awk '{print $3}')

# timing status
if grep -q "slack (VIOLATED)" "$ARCHIVE_DIR/timing_${tag}.rpt"
then
    status="VIOLATED"
else
    status="MET"
fi

printf "%.1f,%s,%s,%s,%s\n" "$p" "$arrival" "$slack" "$status" "$area" >> "$RESULTS"

if [ "$status" = "VIOLATED" ]
then
    printf "Timing violation occurred at %.1f ns\n" "$p"
    break
fi

p=$(awk "BEGIN {printf \"%.1f\", $p - $step}")
done

echo "Sweep complete. Results saved in $RESULTS"
echo "All archived reports saved in $ARCHIVE_DIR"
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

```